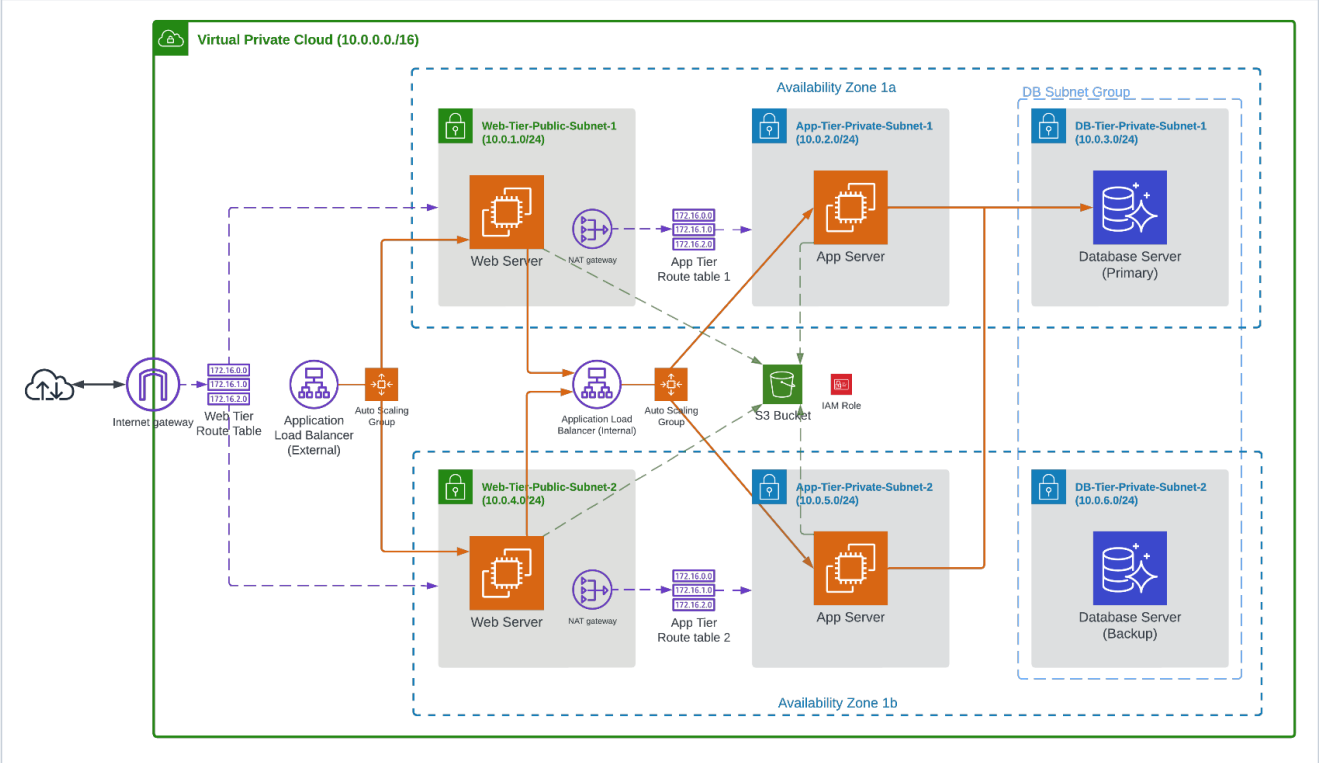
DEPLOY A THREE TIER WEB APPLICATION ON AWS

Created by Rutuja Naik
linkedin.com/in/rutuja-naik07/

From zero to a live, auto-scaling AWS app in 15 steps.



The three-tier web application architecture is a robust and scalable design pattern that separates an application into three distinct layers: the presentation (web) tier, the application tier, and the database tier. It improves security, manageability and flexibility by isolating each layer's responsibilities.

In this guide you will deploy a three-tier web application on AWS, step by step. Beginners can follow every click; experienced engineers can use the tables, commands and checkpoints as a quick reference.

Contents

Contents . . . 2

Introduction . . . 3

Project overview . . . 3

Architecture . . . 4

Project implementation . . . 5

1. Create the VPC and subnets . . . 6

2. Create the internet gateway, NAT gateways and route tables . . . 10

3. Create the security groups . . . 15

4. Create the database subnet group and RDS . . . 16

5. Create the IAM role . . . 22

6. Create an S3 bucket and upload the source code . . . 23

7. Create the test app server and set it up . . . 24

8. Create an AMI of the app server . . . 34

9. Create the target group, launch template and internal load balancer (app tier) . . . 34

10. Create the test web server and set it up . . . 40

11. Create an AMI of the web server . . . 41

12. Create the target group, launch template and external load balancer (web tier) . . . 41

13. Create the Auto Scaling groups . . . 42

14. Access the application . . . 45

15. Clean up the infrastructure . . . 47

Quick troubleshooting . . . 48

Your journey at a glance

Four phases, fifteen checkpoints. Each step ends with a quick check, so you always know you are on track.



Introduction

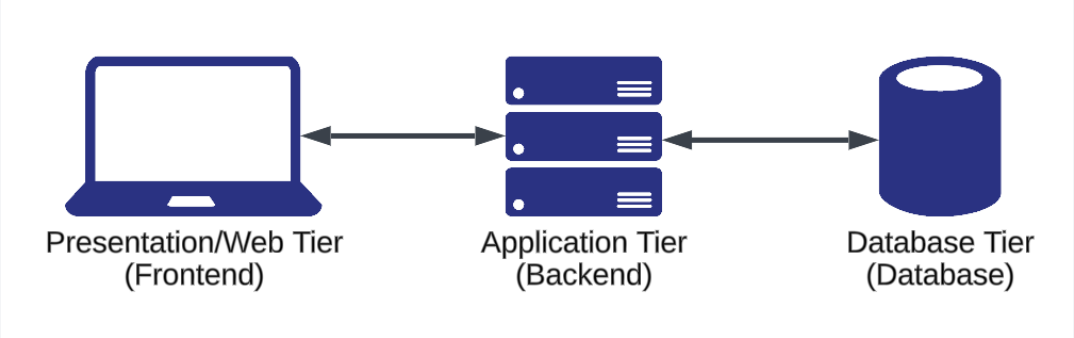
15 guided steps	**3** tiers	**2** Availability Zones	**14** AWS services

Ever wondered how real apps stay online even when a server fails? This is how. You will build a small but production-style app where the website, the logic and the data each live in their own protected layer, and AWS keeps them healthy for you. By the end, you will have something you can open in a browser, share, and explain with confidence.

A three-tier architecture is a common pattern in software development and deployment, especially in cloud environments like AWS. It separates an application into three main layers.

1. **Presentation / Web tier (frontend).** The user interface layer where users interact with the application. It presents information and collects user input.
2. **Application tier (backend).** The logic layer where processing and control flow happen. It processes user input, performs calculations and makes decisions.
3. **Database tier.** The layer responsible for data storage and management. It stores, retrieves and manages the data the application tier needs.

Users interact with the application through the presentation tier. Their requests go to the application tier, where the logic processes them. The application tier talks to the database tier to retrieve or store data. The result goes back to the presentation tier, which shows it to the user.



Project overview

In this project you build a **highly available, highly secured, highly scalable and fault-tolerant** three-tier application on AWS. You will work with these components:

1	Amazon Elastic Compute Cloud (EC2)	8	Amazon Machine Image (AMI)
2	Amazon Simple Storage Service (S3)	9	Launch Templates
3	Virtual Private Cloud (VPC)	10	Target Groups
4	Auto Scaling Group (ASG)	11	Internet Gateway (IGW)

5	Application Load Balancer (ALB)	12	NAT Gateway
6	Identity and Access Management (IAM)	13	Route Table
7	Amazon Relational Database Service (RDS)	14	Security Groups

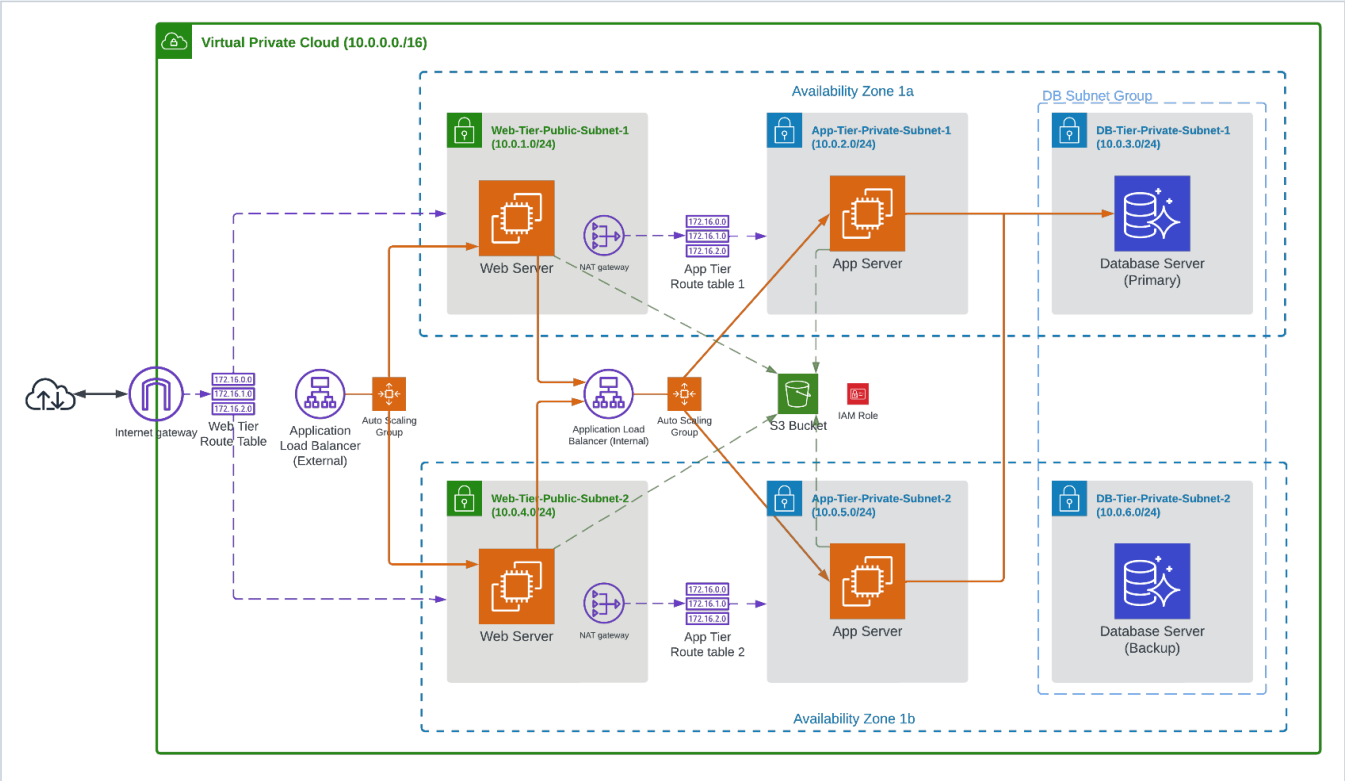


Key terms in plain English

Term	What it means
VPC	Your own private network inside AWS.
Subnet	A slice of the VPC. Public subnets can reach the internet directly; private subnets cannot be reached from it.
Internet gateway / NAT gateway	The internet gateway is the door to the internet. A NAT gateway lets private servers go out (for downloads) without letting anyone in.
Route table	Rules that say where network traffic goes.
Security group	A firewall around a resource.
IAM role	Permissions a server can use, so no passwords are stored on it.
AMI and launch template	An AMI is a saved copy of a configured server. A launch template is the recipe for new servers.
Target group, load balancer, Auto Scaling group	A target group is the list of servers a load balancer sends traffic to. The Auto Scaling group keeps the right number of healthy servers running.

Architecture

The diagram below shows the overview of the project: how each component connects, and how everything is distributed across two Availability Zones.



Architecture diagram: VPC 10.0.0.0/16 with public web subnets, private app subnets and a private database subnet group in two Availability Zones.

How a request travels:



Orange = reachable from the internet (public subnets). Navy = private.

We use **public subnets** for the web tier because it must be open to the internet so users can reach the application. The app tier and database tier must be secured, so they use **private subnets**. Resources are spread across two Availability Zones, and everything lives in our custom VPC. Notice how a CIDR block is assigned to each subnet.

An **S3 bucket** stores the source code of the application. This makes server configuration much easier.

> **Note.** Source code: https://github.com/Rutuja-Naik07/Project-Three-Tier-Architecture-on-AWS.git

> **Heads up.** Several services here bill by the hour (NAT gateways, load balancers, the database). Finish the project, then follow Step 15 to delete everything.

Project implementation

Go to the **AWS Console** and choose a region you like (the original guide uses Mumbai, ap-south-1). Use the same region for everything. To open each service, use the search bar and search for it.

> **Note.** When naming components you can use any names you like. Keep them consistent so you can find things later.

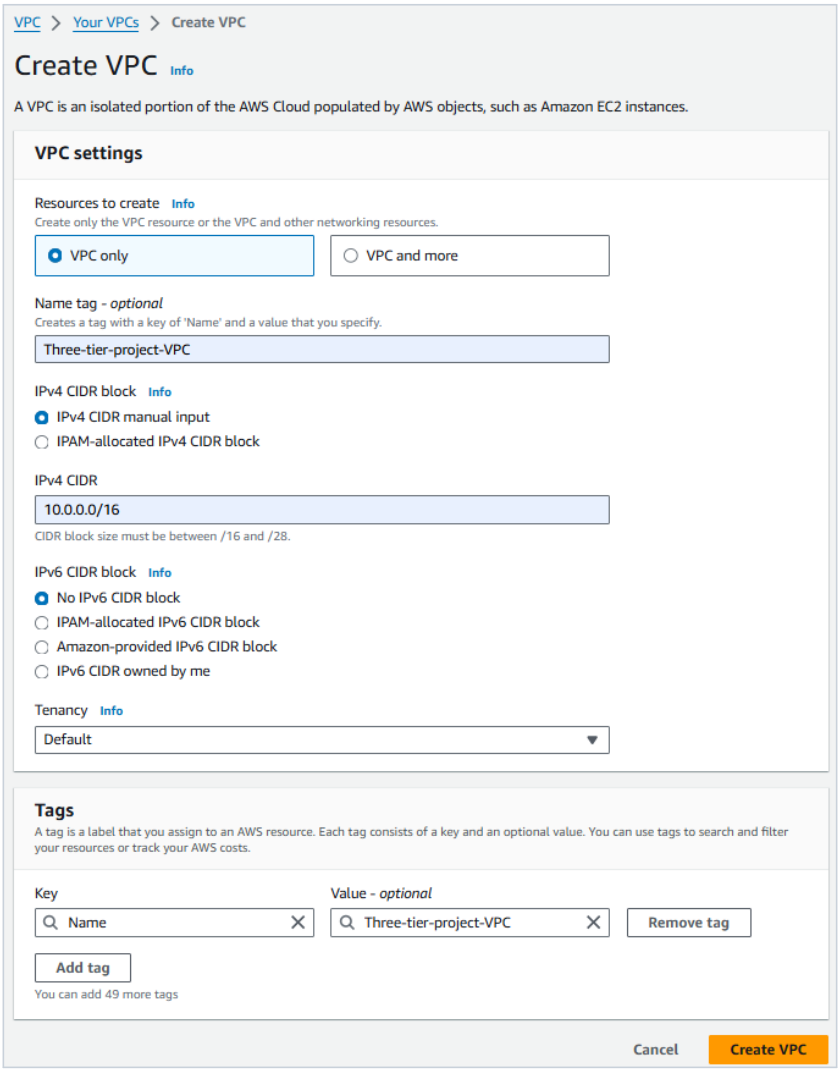
1 Create the VPC and subnets

Build the neighborhood

> **Why bother?.** Everything else lives inside this private network, so we build it first.

Create the VPC

Search for **VPC** and go to **Your VPCs > Create VPC**. Set it up as shown. Use **10.0.0.0/16** as the CIDR block.



Create VPC: choose VPC only, name it Three-tier-project-VPC, IPv4 CIDR 10.0.0.0/16.

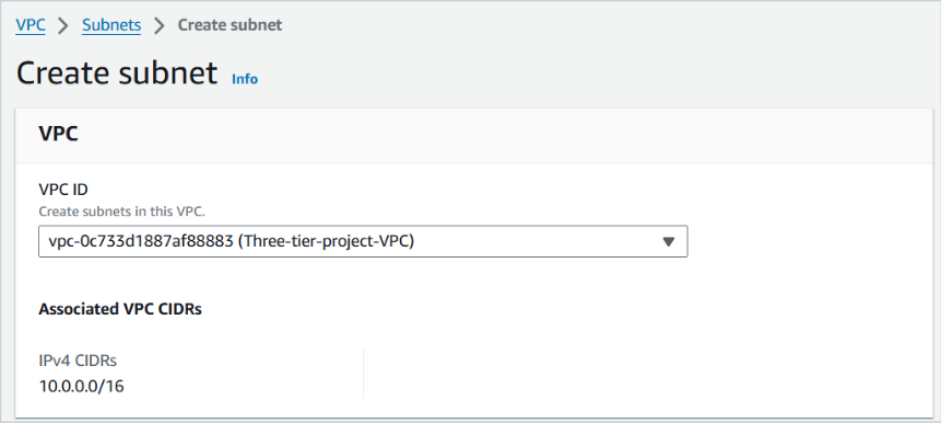
Create the subnets

Next, create six subnets in this VPC (refer to the architecture diagram).

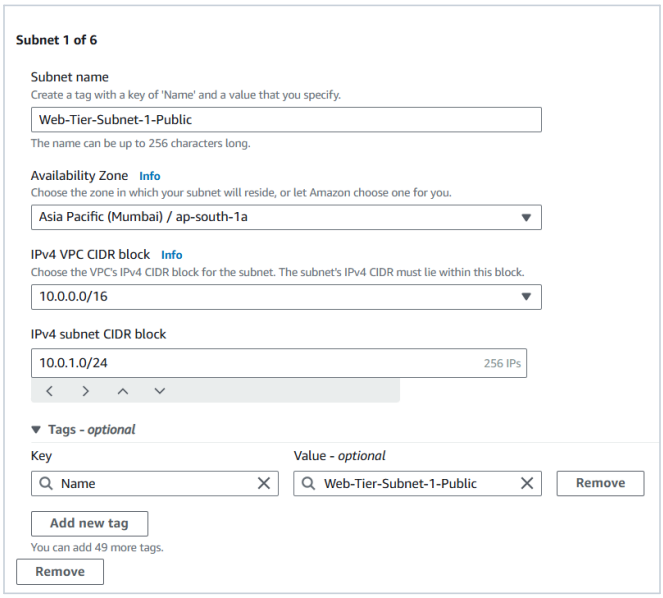
Availability Zone 1a	Availability Zone 1b
1. Web-Tier-Subnet-1-Public 2. App-Tier-Subnet-1-Private 3. DB-Tier-Subnet-1-Private	4. Web-Tier-Subnet-2-Public 5. App-Tier-Subnet-2-Private 6. DB-Tier-Subnet-2-Private

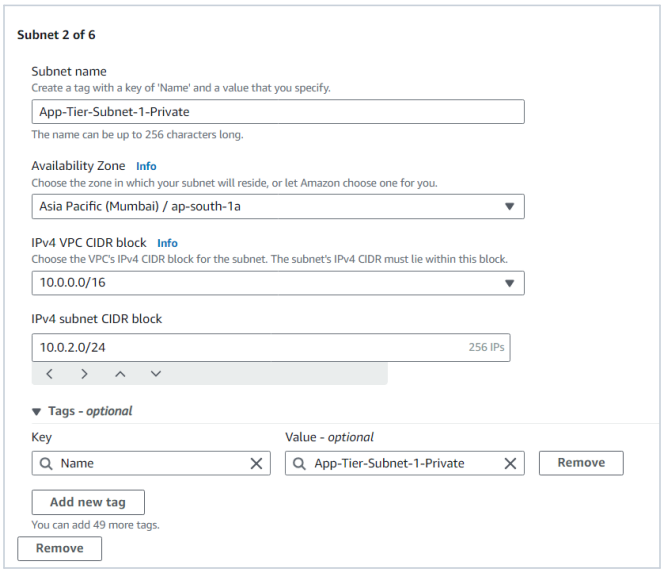
Subnet	Zone	Type	CIDR
Web-Tier-Subnet-1-Public	1a	Public	10.0.1.0/24
App-Tier-Subnet-1-Private	1a	Private	10.0.2.0/24
DB-Tier-Subnet-1-Private	1a	Private	10.0.3.0/24
Web-Tier-Subnet-2-Public	1b	Public	10.0.4.0/24
App-Tier-Subnet-2-Private	1b	Private	10.0.5.0/24
DB-Tier-Subnet-2-Private	1b	Private	10.0.6.0/24

Go to **VPC dashboard > Subnets > Create subnet**, select your VPC, and set up each subnet as follows. Then click **Create subnet**.

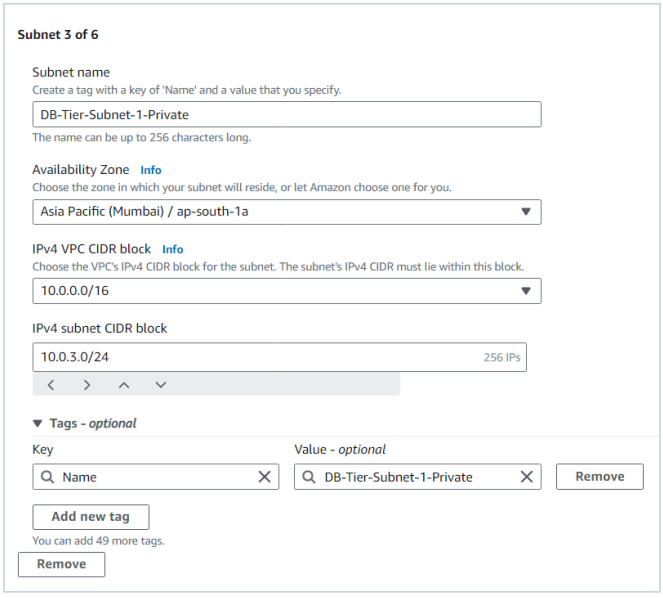


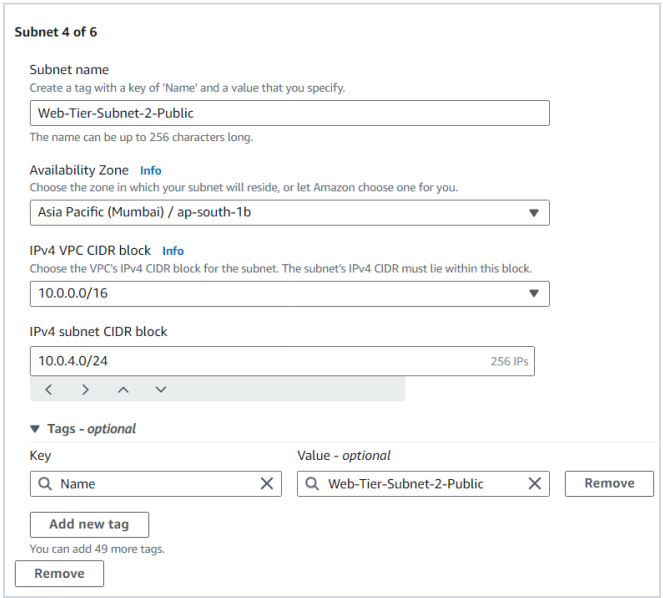
Create subnet: choose the Three-tier-project-VPC.



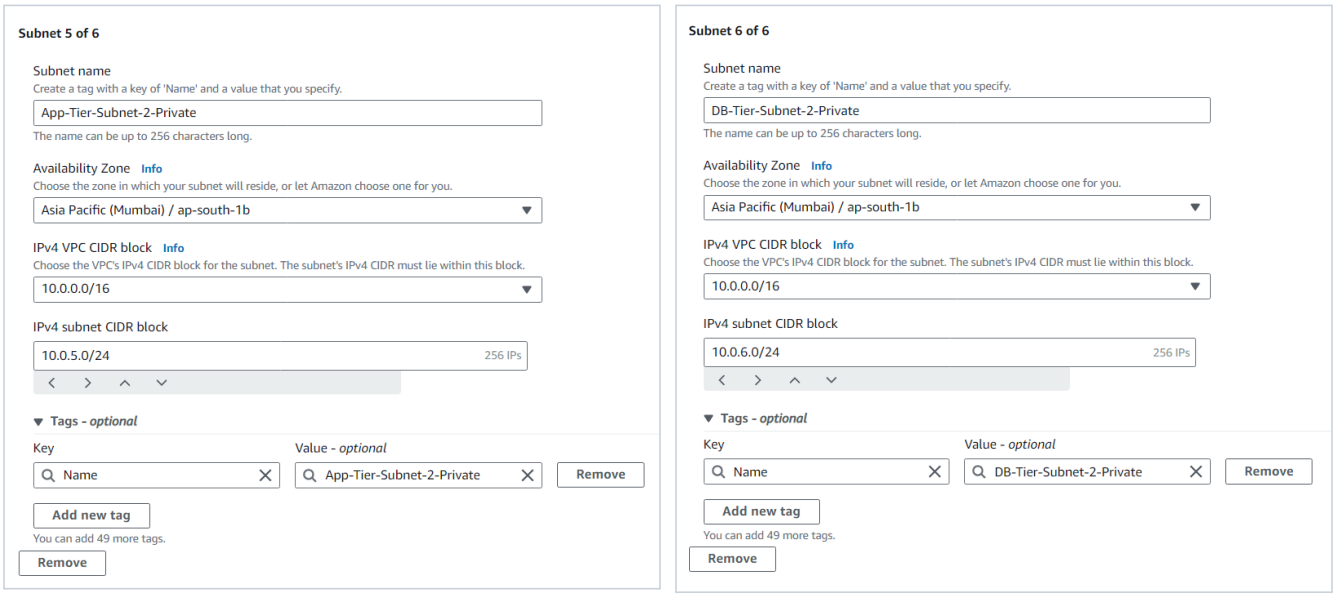


Subnet 1 of 6 (Web-Tier-Subnet-1-Public, 10.0.1.0/24) and Subnet 2 of 6 (App-Tier-Subnet-1-Private, 10.0.2.0/24), both in ap-south-1a.



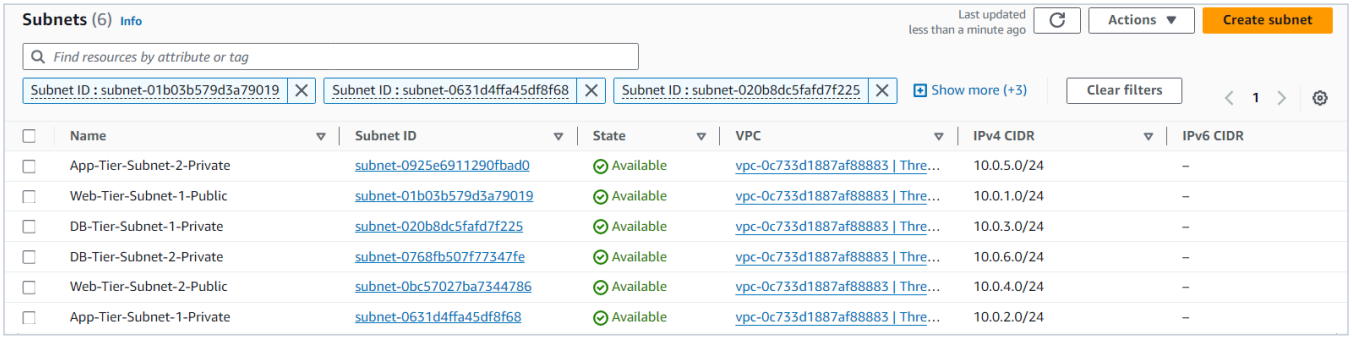


Subnet 3 of 6 (DB-Tier-Subnet-1-Private, 10.0.3.0/24, 1a) and Subnet 4 of 6 (Web-Tier-Subnet-2-Public, 10.0.4.0/24, 1b).



Subnet 5 of 6 (App-Tier-Subnet-2-Private, 10.0.5.0/24) and Subnet 6 of 6 (DB-Tier-Subnet-2-Private, 10.0.6.0/24), both in 1b.

They will be created as follows:



All six subnets, available, with the CIDR ranges from the plan.

Now select **Web-Tier-Subnet-1-Public**. Go to **Actions > Edit subnet settings**. Under auto-assign IP settings, select **Enable auto-assign public IPv4 address** and save. Do the same for **Web-Tier-Subnet-2-Public**.

> **Checkpoint.** Six subnets exist in your VPC, and both web subnets auto-assign public IPs.

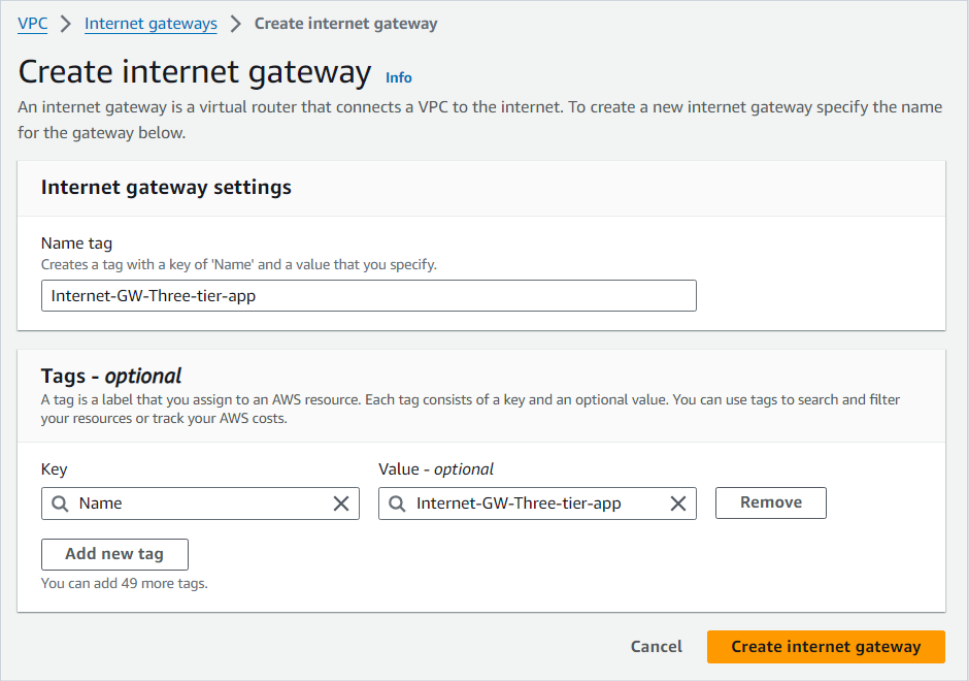
2 Create the internet gateway, NAT gateways and route tables

Open the doors and lay the roads

Why bother?. The web tier needs a path to the internet. Private servers need a way out for downloads, without letting anyone in.

Create the internet gateway

Go to **VPC dashboard > Internet gateways > Create internet gateway**. Enter a name and click create.

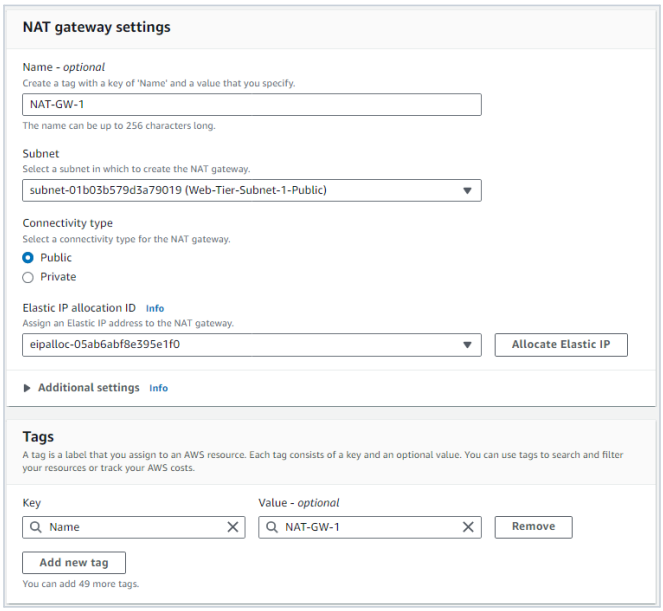


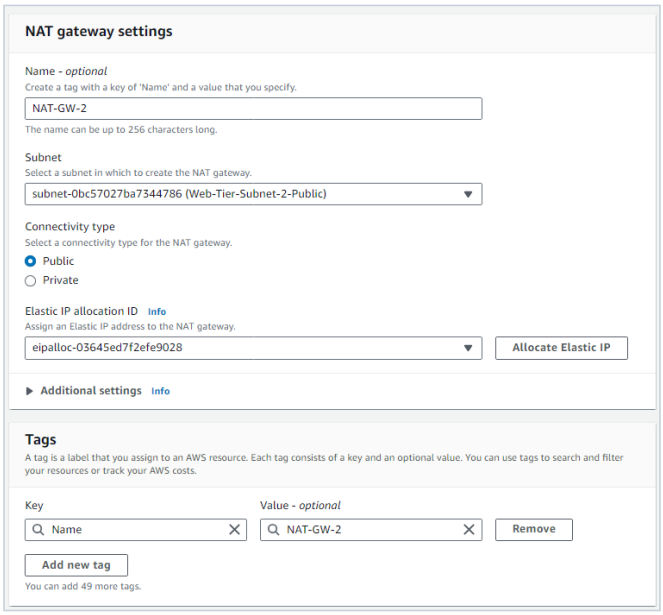
Create internet gateway: name tag Internet-GW-Three-tier-app.

Then open the internet gateways list, select your gateway and go to **Actions > Attach to VPC**. Select your VPC.

Create the NAT gateways

Go to **VPC dashboard > NAT gateways > Create NAT gateway**. Create two: **NAT-GW-1** in the public web subnet in 1a and **NAT-GW-2** in the public web subnet in 1b. Allocate an Elastic IP for each.

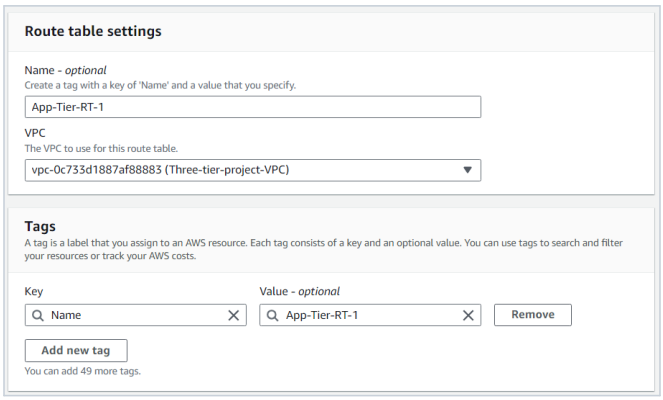


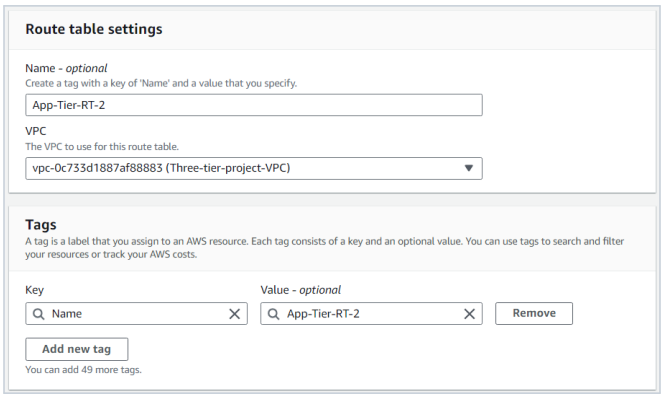


NAT-GW-1 in Web-Tier-Subnet-1-Public (left) and NAT-GW-2 in Web-Tier-Subnet-2-Public (right). Connectivity type: Public.

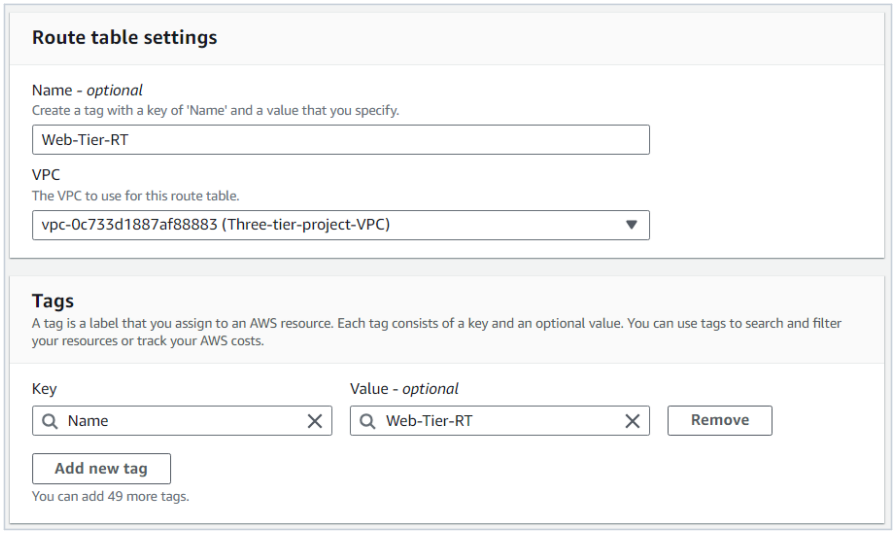
Create the route tables

According to the diagram, create three route tables: **App-Tier-RT-1**, **App-Tier-RT-2** and **Web-Tier-RT**. Go to **VPC dashboard > Route tables > Create route table** and create them as shown.



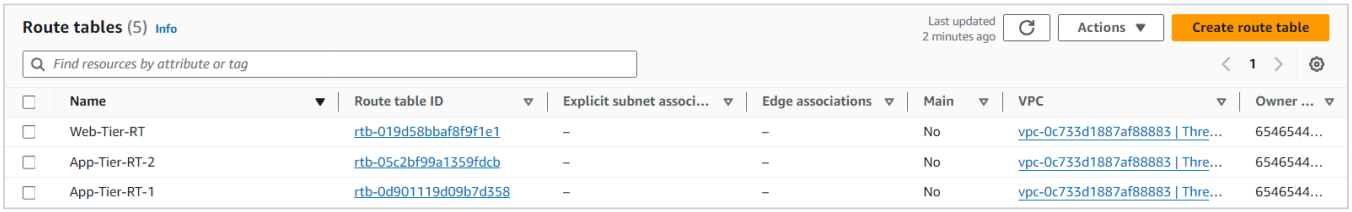


App-Tier-RT-1 (left) and App-Tier-RT-2 (right), both in Three-tier-project-VPC.



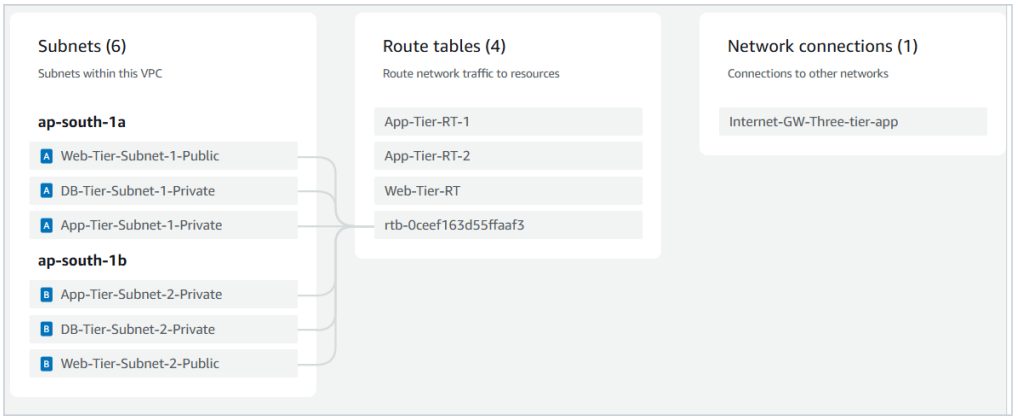
Web-Tier-RT.

Now you will see all the route tables in the menu.



The route tables list.

Go to **VPC dashboard > Your VPCs**, select Three-tier-project-VPC and open **Resource map**. The route tables exist, but there are no connections yet.

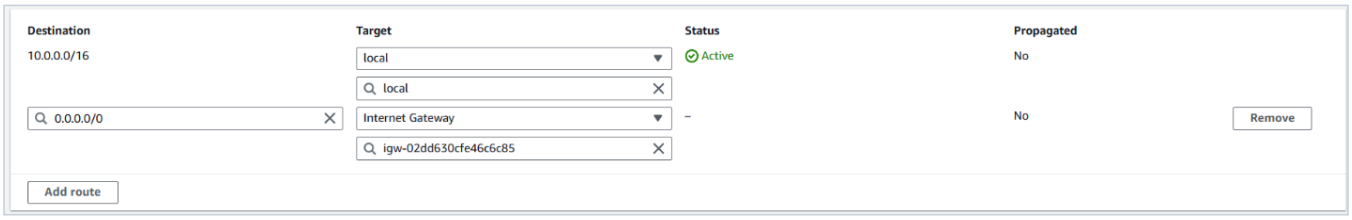


Resource map before any routes or associations.

Edit routes and subnet associations

Route table	Add route: 0.0.0.0/0 to	Associate with subnet
Web-Tier-RT	Internet gateway (your igw)	Web-Tier-Subnet-1-Public and Web-Tier-Subnet-2-Public
App-Tier-RT-1	NAT gateway NAT-GW-1	App-Tier-Subnet-1-Private
App-Tier-RT-2	NAT gateway NAT-GW-2	App-Tier-Subnet-2-Private

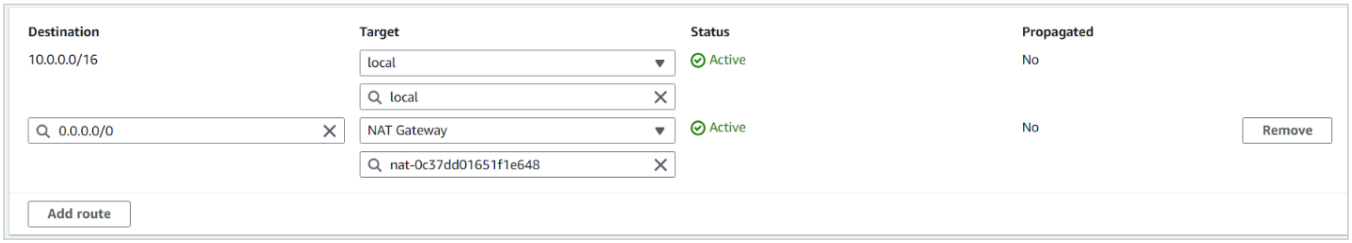
1. Select **Web-Tier-RT**, go to **Actions > Edit routes**. Add route: destination 0.0.0.0/0, target Internet Gateway, then select your igw. Save changes.



Web-Tier-RT: 0.0.0.0/0 points to the Internet Gateway.

Select **Web-Tier-RT** again and go to **Actions > Edit subnet associations**. Select Web-Tier-Subnet-1-Public and Web-Tier-Subnet-2-Public and save the associations.

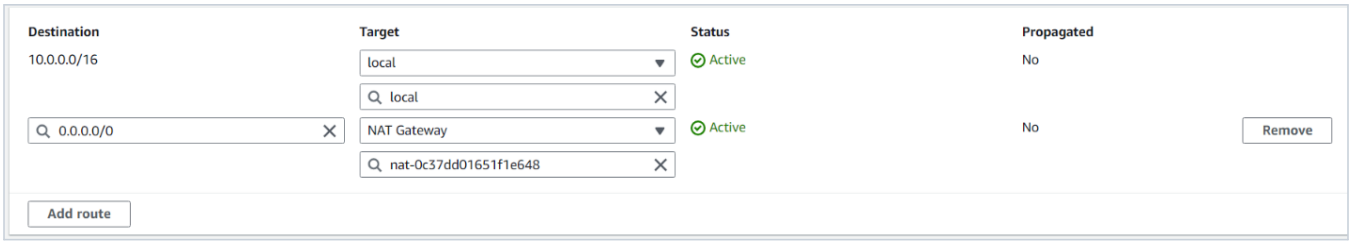
2. Select **App-Tier-RT-1**, go to **Actions > Edit routes**. Add route: destination 0.0.0.0/0, target NAT Gateway, then select NAT-GW-1. Save changes.



App-Tier-RT-1: 0.0.0.0/0 points to NAT-GW-1.

Select **App-Tier-RT-1** again and go to **Actions > Edit subnet associations**. Select App-Tier-Subnet-1-Private and save the associations.

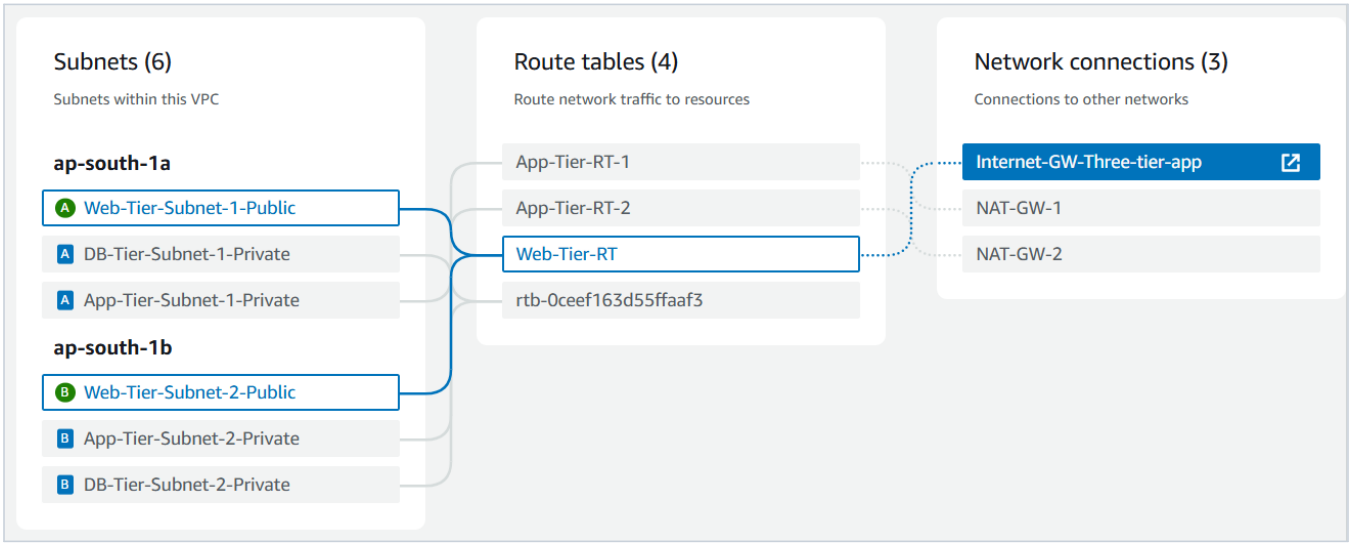
3. Select **App-Tier-RT-2**, go to **Actions > Edit routes**. Add route: destination 0.0.0.0/0, target NAT Gateway, then select NAT-GW-2. Save changes.



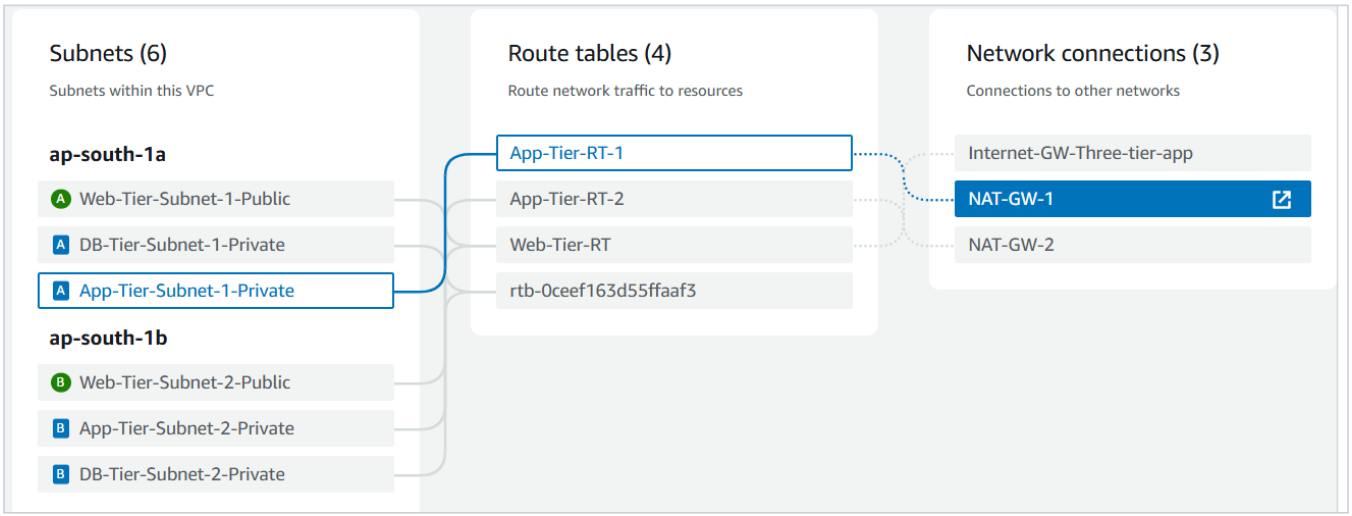
App-Tier-RT-2: 0.0.0.0/0 points to the NAT gateway.

Select **App-Tier-RT-2** again and go to **Actions > Edit subnet associations**. Select App-Tier-Subnet-2-Private and save the associations.

Now check the Resource map. The connections have been made. Look for mistakes against the architecture diagram by hovering over each element.



Resource map: Web-Tier-RT connects both public web subnets to the internet gateway.



Resource map: App-Tier-RT-1 connects App-Tier-Subnet-1-Private to NAT-GW-1.

Pro tip. Hover over each item in the Resource map to see exactly what it connects to. It is the fastest way to catch a wrong association.

Checkpoint. Every route table is connected to its subnets in the Resource map. Good job. Now let's create the security groups.

3 Create the security groups

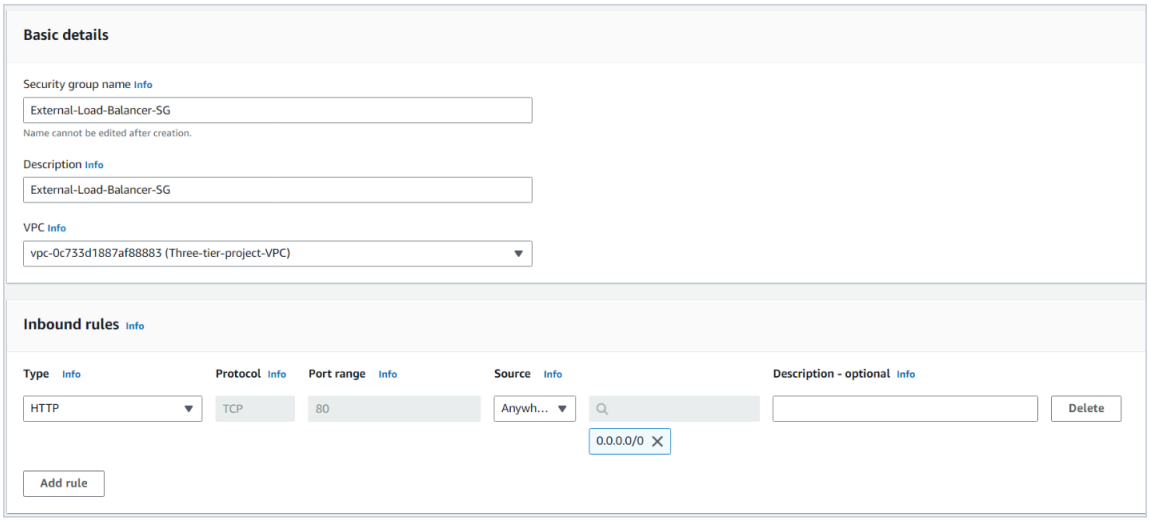
Hire the bouncers

Why bother?. Each tier accepts traffic only from the tier in front of it, so the database can never be reached from the internet.

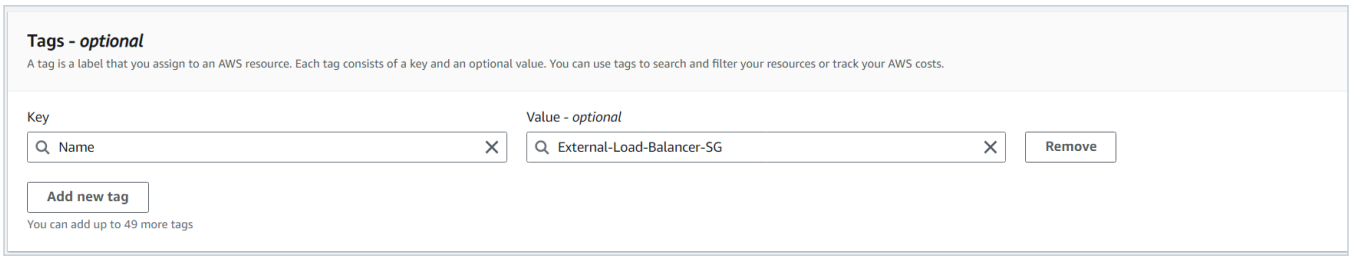
Create the following security groups.

Security group	Type (port range)	Source
External-Load-Balancer-SG	HTTP (80)	0.0.0.0/0
Web-Tier-SG	HTTP (80)	External-Load-Balancer-SG
Internal-Load-Balancer-SG	HTTP (80)	Web-Tier-SG
App-Tier-SG	Custom TCP (4000)	Internal-Load-Balancer-SG
DB-Tier-SG	MYSQL (3306)	App-Tier-SG

Go to **VPC dashboard > Security groups > Create security group**. Create **External-Load-Balancer-SG** and add the inbound rule by referring to the table above. Then click **Create security group**.

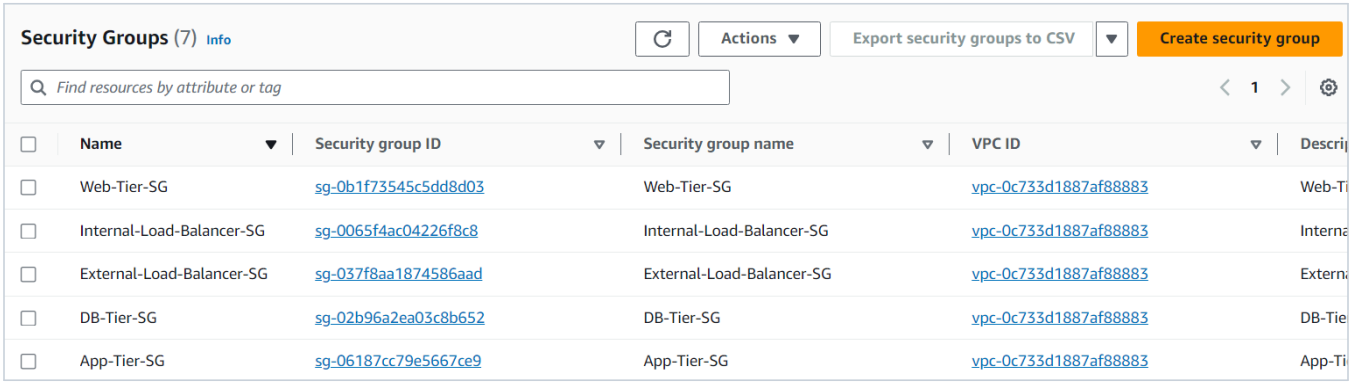


External-Load-Balancer-SG: basic details and the inbound HTTP rule from 0.0.0.0/0.



The Name tag for the security group.

Create the other security groups from the table in the same way. When you are done you can see them like this.



All security groups created.

Checkpoint. Five new security groups exist in your VPC. Well done. Now let's set up the database servers.

Level up. Level 1 complete. Your network, routes and firewalls are ready. Next up: data and keys.

Step 4 of 15 | Data and keys

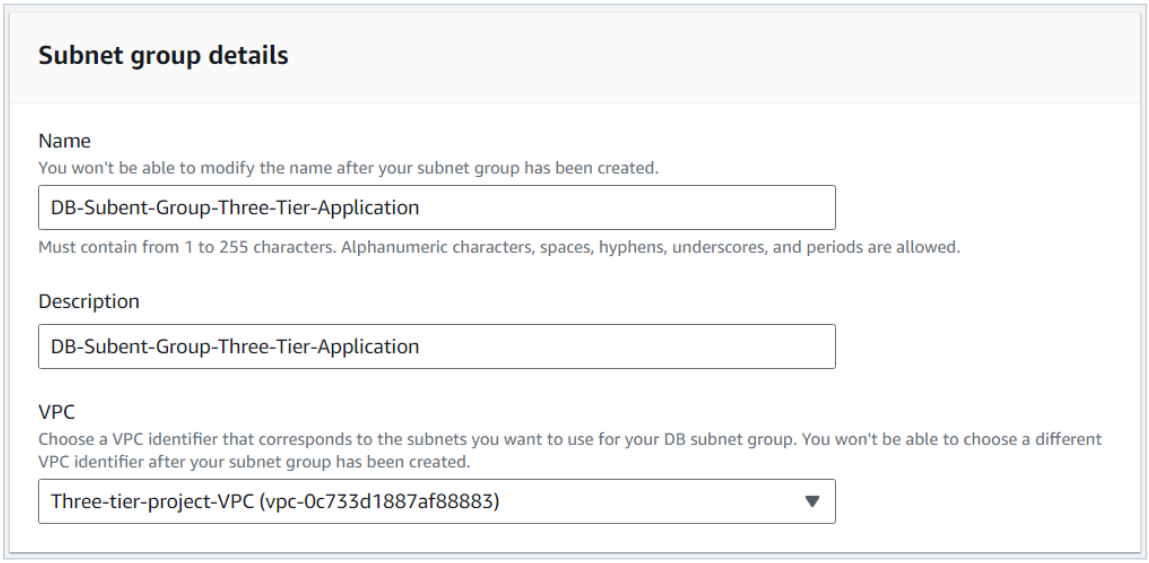
4 Create the database subnet group and RDS

Give your data a safe home

First, we need a subnet group. But why?

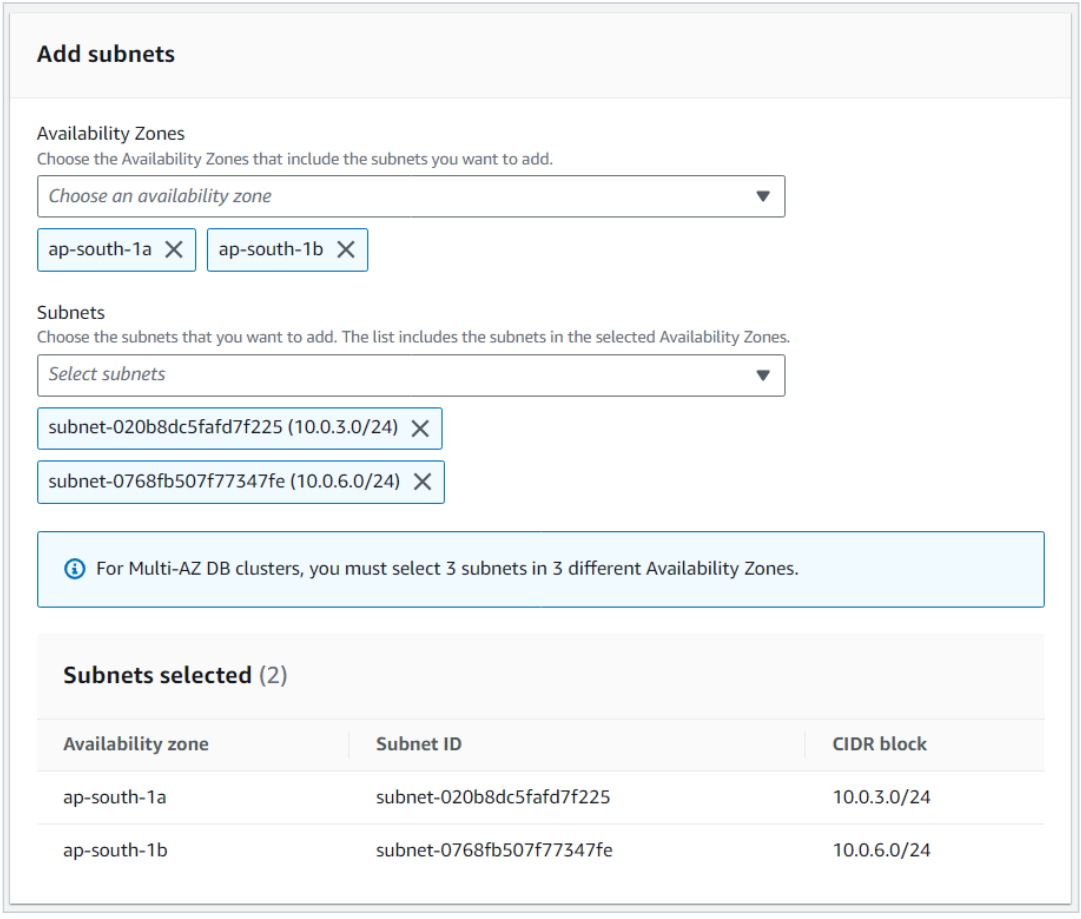
Why bother?. When you deploy an RDS instance in a Multi-AZ configuration, AWS automatically creates a primary database instance in one Availability Zone and a standby replica in another AZ in the same region. This requires a subnet group, because each subnet in the group must belong to a different AZ. That spreads your database across multiple AZs for redundancy and failover protection.

Search for **RDS**. In Amazon RDS go to **Subnet groups > Create DB subnet group** and set it up as follows.



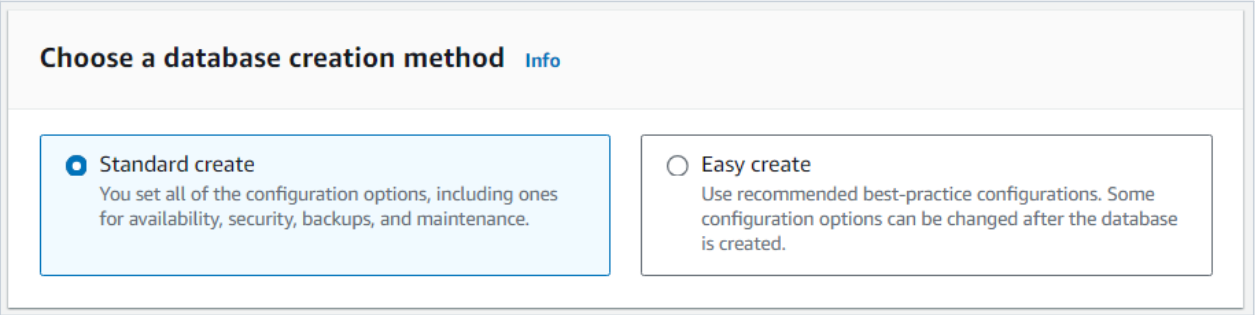
DB subnet group details: name, description and VPC.

Choose the Availability Zones from the diagram (1a, 1b). Choose the subnets assigned to the database (10.0.3.0/24 and 10.0.6.0/24).

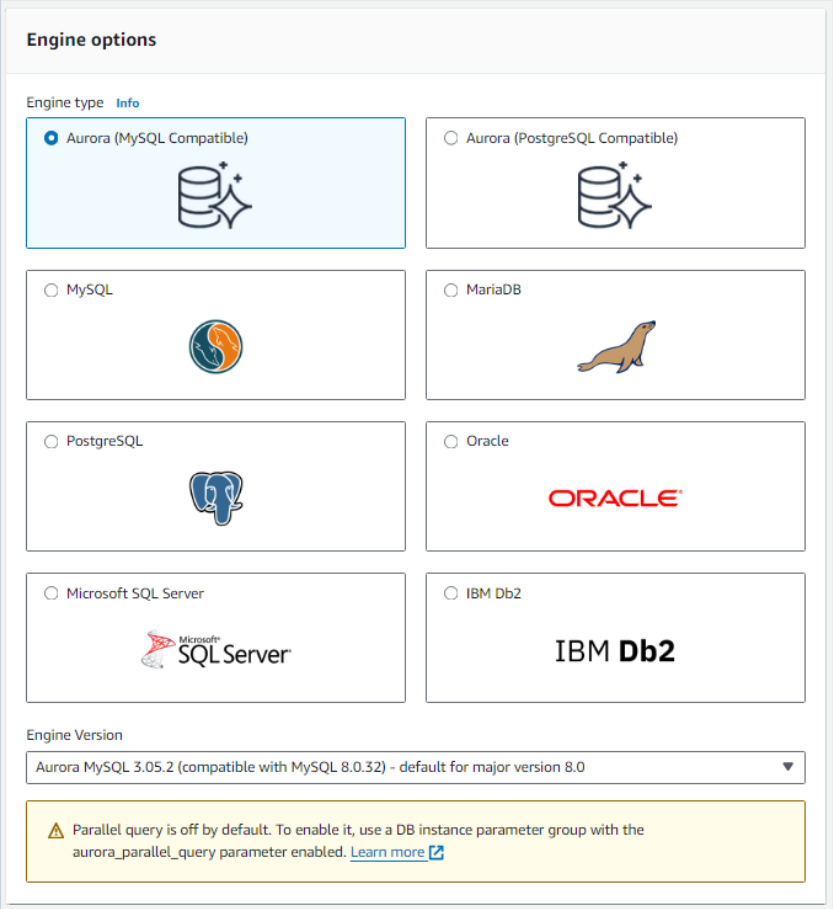


Adding the two DB subnets, one in each Availability Zone.

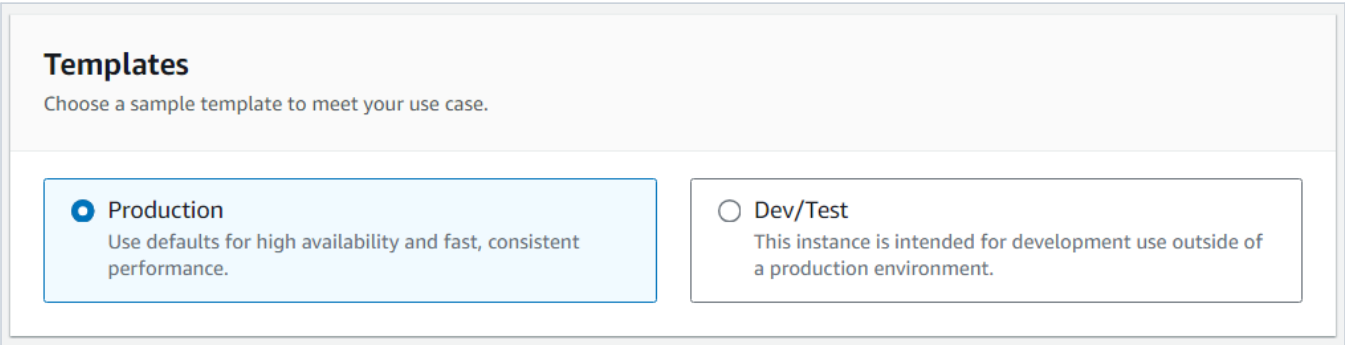
Now go to **Amazon RDS > Databases > Create database** and set it up as follows. The application uses MySQL, so choose **Aurora (MySQL Compatible)**. You can use plain MySQL as well.



Choose a database creation method: Standard create.

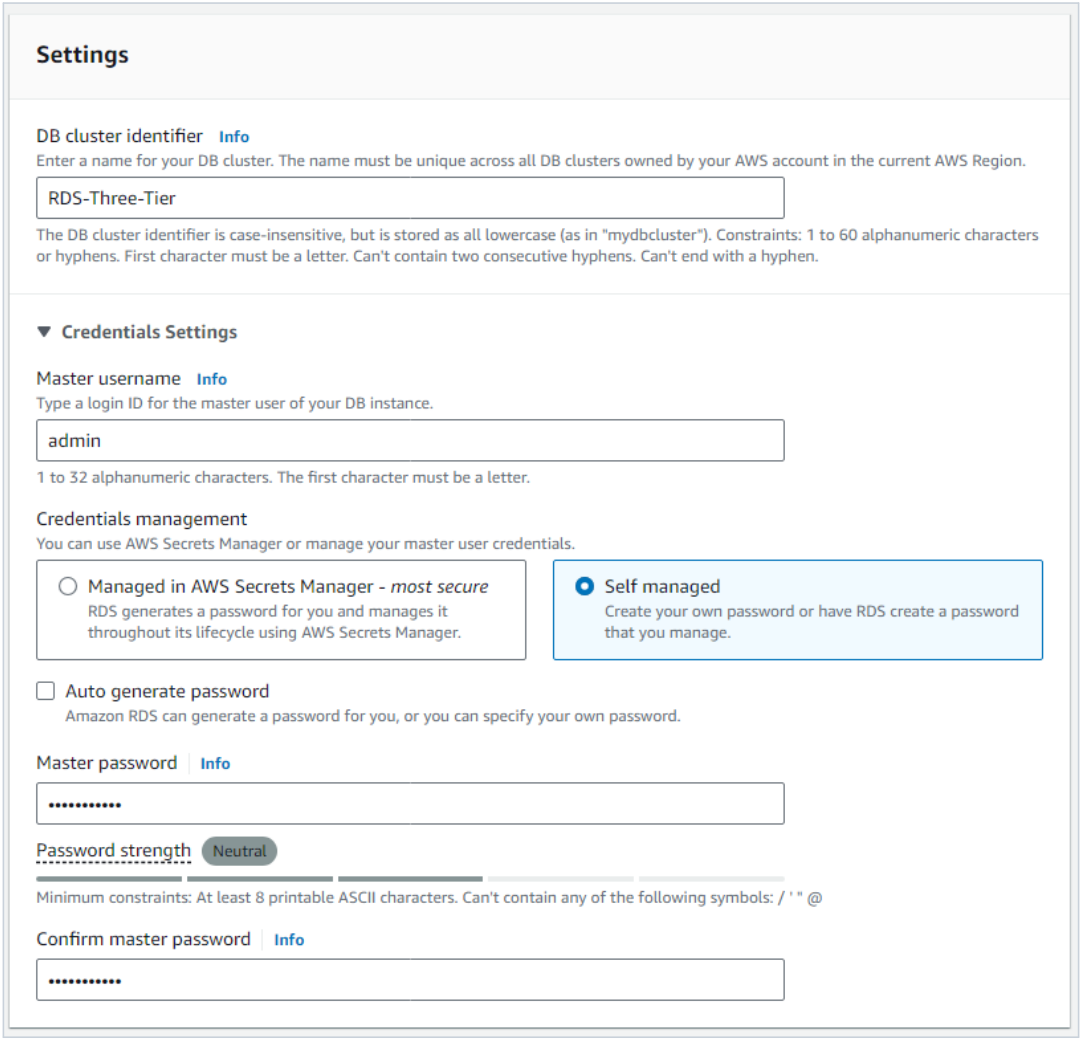


Engine options: Aurora (MySQL Compatible).

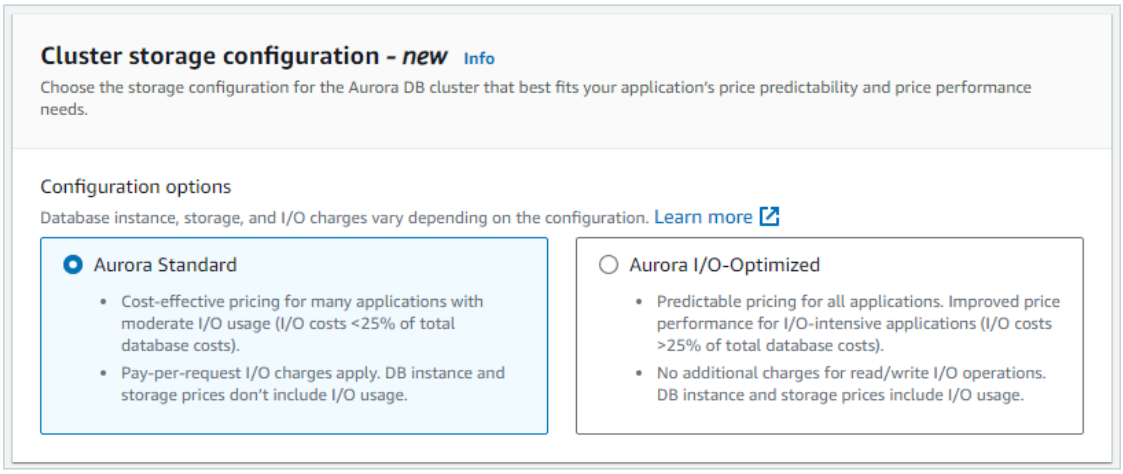


Template: Production.

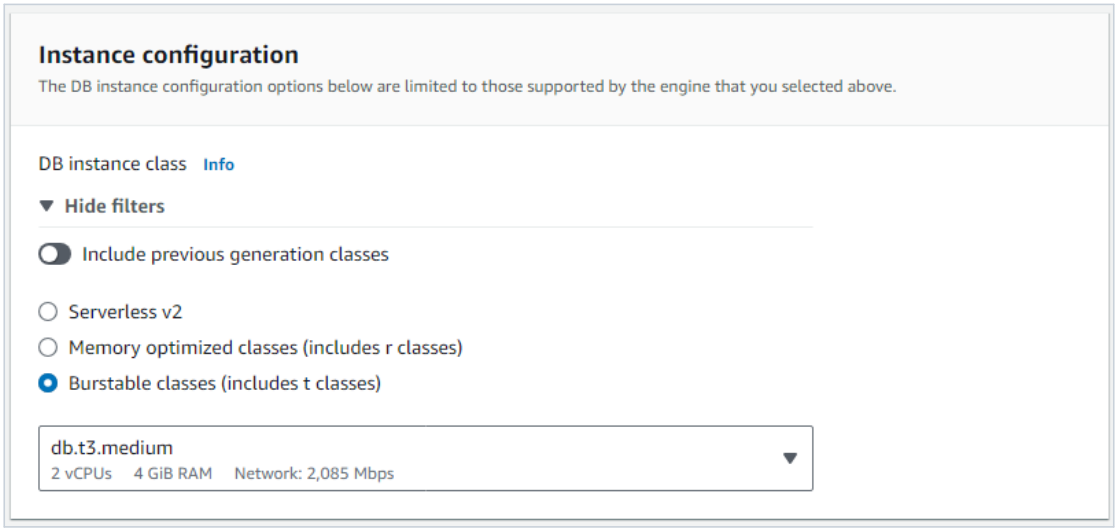
Heads up. Remember the master username and password. You need them when you access the database server in later steps. Choose your own strong password. Never reuse the demo password shown in the original screenshots.



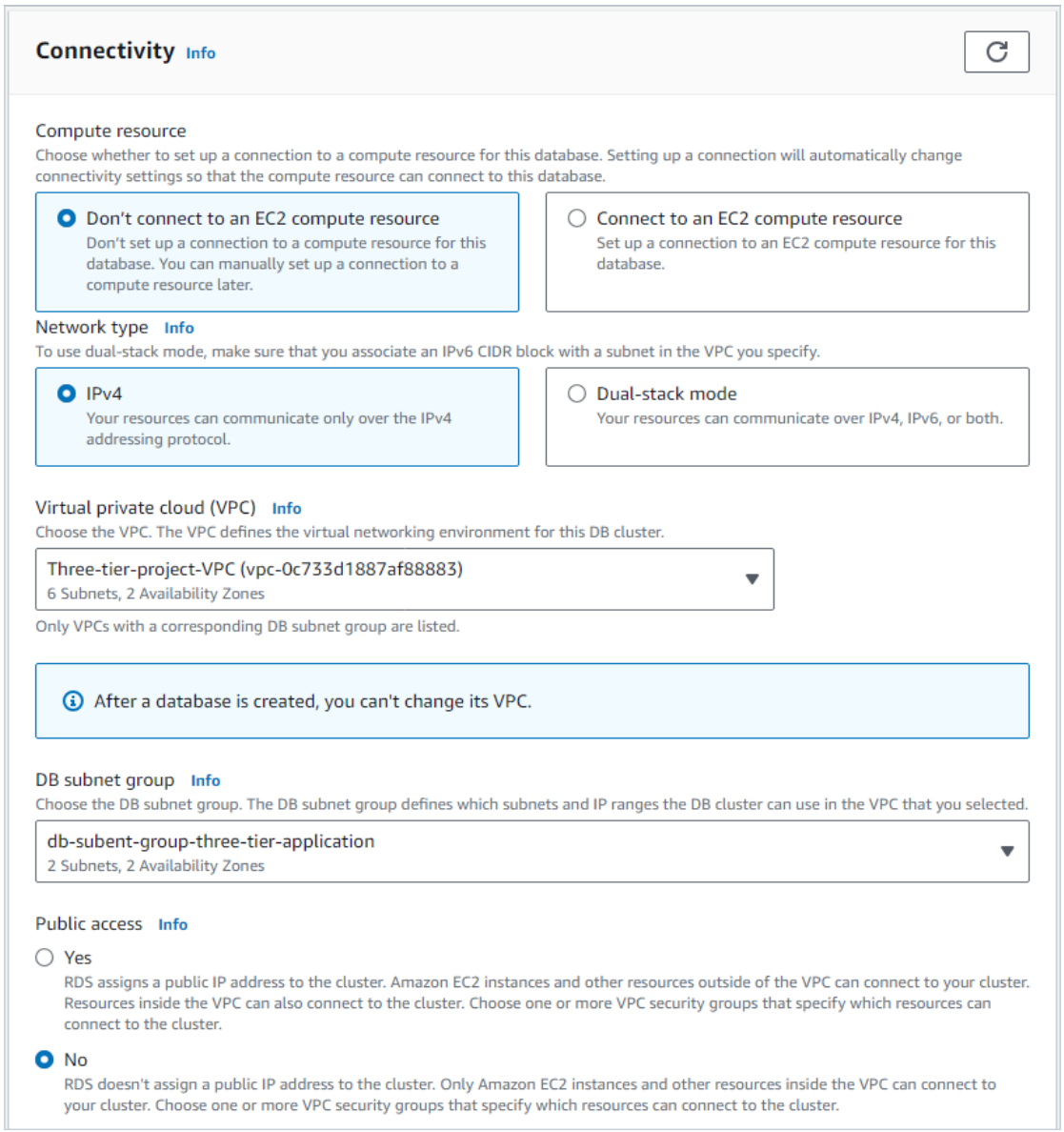
Settings: DB cluster identifier, master username (admin) and your own password.



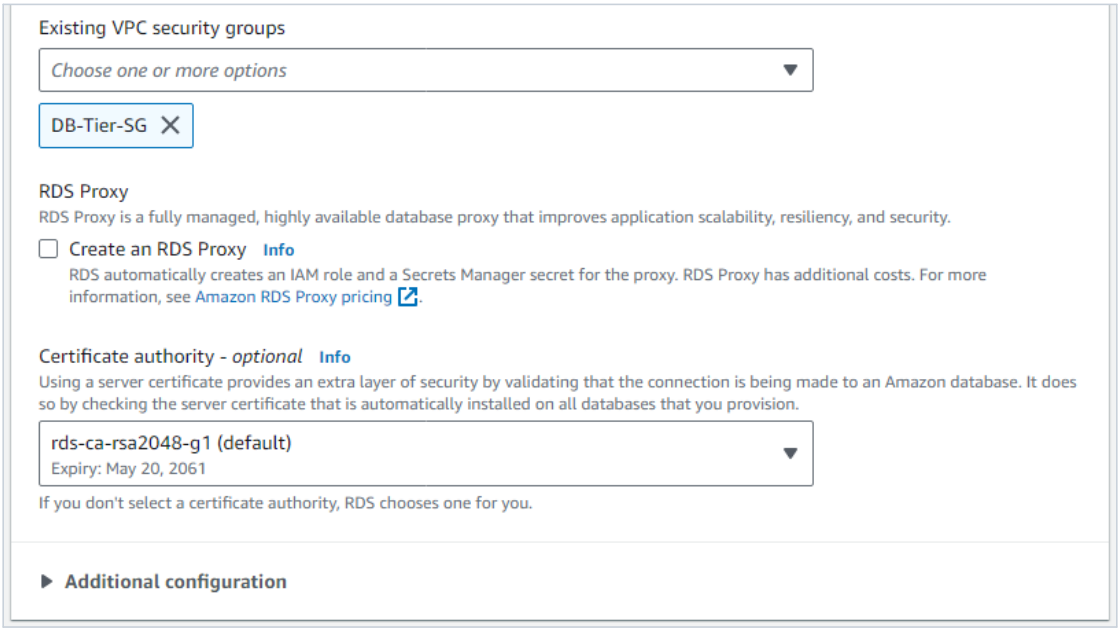
Cluster storage configuration: Aurora Standard.



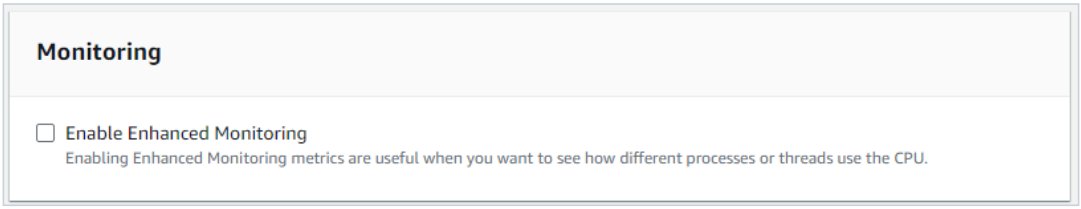
Instance configuration: Burstable class db.t3.medium.



Connectivity: your VPC, your DB subnet group, and Public access set to No.

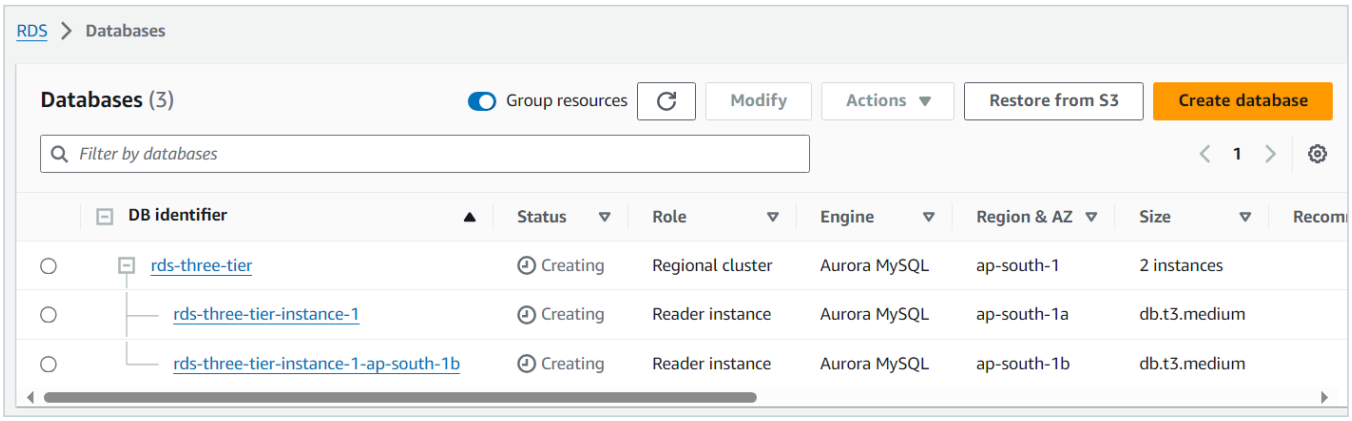


Existing VPC security group: DB-Tier-SG.



Monitoring: Enhanced Monitoring left off.

Keep the other settings as default. This database costs some money, but since you delete it as soon as the project is done, it will be a few dollars. Now click **Create database**. It takes some time to create the server.



The new cluster, in the Creating state.

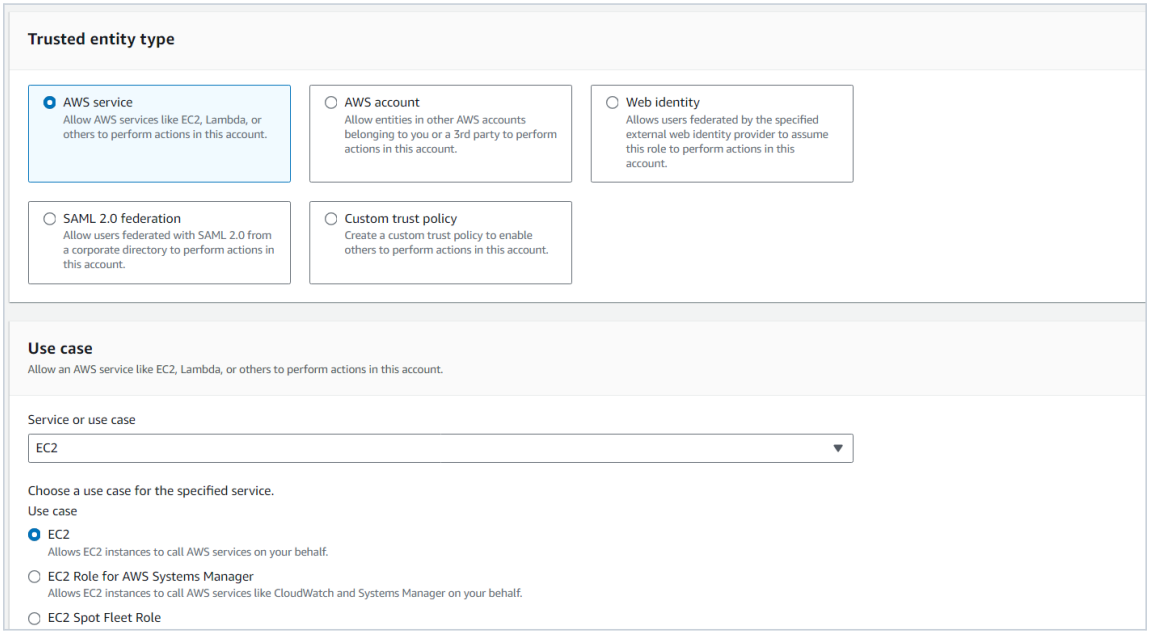
Checkpoint. The database status is Available. Great, the database server is created.

5 Create the IAM role

Hand out the right keys

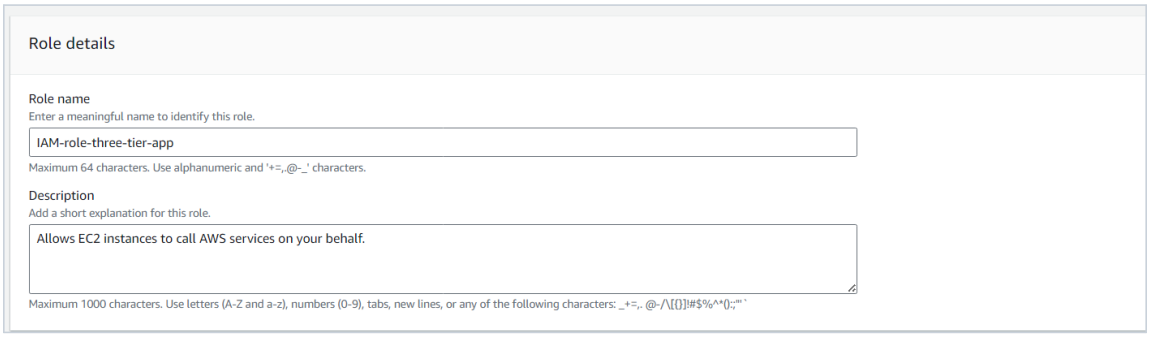
The application should be highly secured. In the IAM role you use two permission policies. **AmazonS3ReadOnlyAccess** lets servers read the S3 bucket. **AmazonSSMManagedInstanceCore** grants an EC2 instance the permissions to communicate with the Systems Manager service.

Search for **IAM**, then go to **Roles > Create role**.

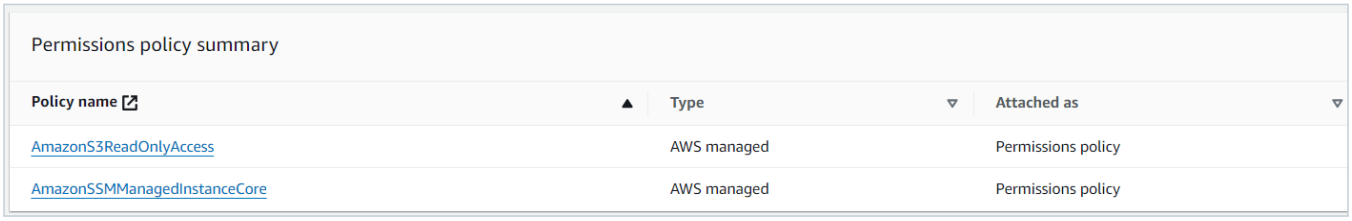


Trusted entity type: AWS service. Use case: EC2.

Click Next. Search for **AmazonS3ReadOnlyAccess** and **AmazonSSMManagedInstanceCore** and select them. Click Next.



Role details: a name such as IAM-role-three-tier-app.



Permissions policy summary: both policies attached.

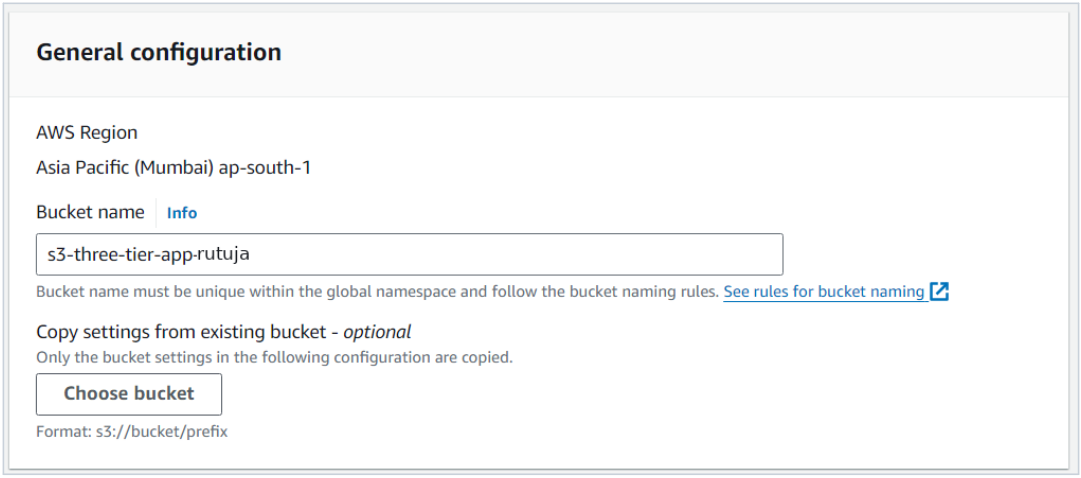
Click **Create role**.

Step 6 of 15 | Data and keys

6 Create an S3 bucket and upload the source code

Stock the code pantry

When you set up the application on servers, it is easier if the source code is in an S3 bucket. Search for **S3** and click **Create bucket**. The bucket name must be unique.

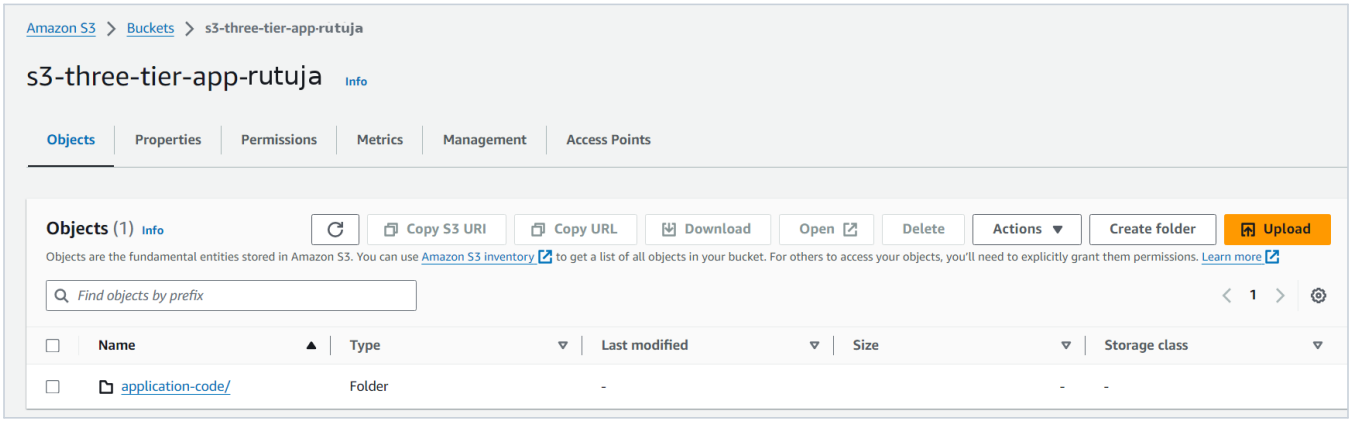


Create bucket: your region and a globally unique name.

Leave the other options as default and click **Create bucket**. Now clone the repository to your PC. It contains all the source code.

```
git clone https://github.com/Rutuja-Naik07/Project-Three-Tier-Architecture-on-AWS.git
```

Go to the bucket and upload the **application-code** folder. You will change some files in later steps.



The bucket after uploading the application-code folder.

Level up. Level 2 complete. The database, permissions and code storage are in place. Time to build the servers.

Step 7 of 15 | Build the tiers

7 Create the test app server and set it up

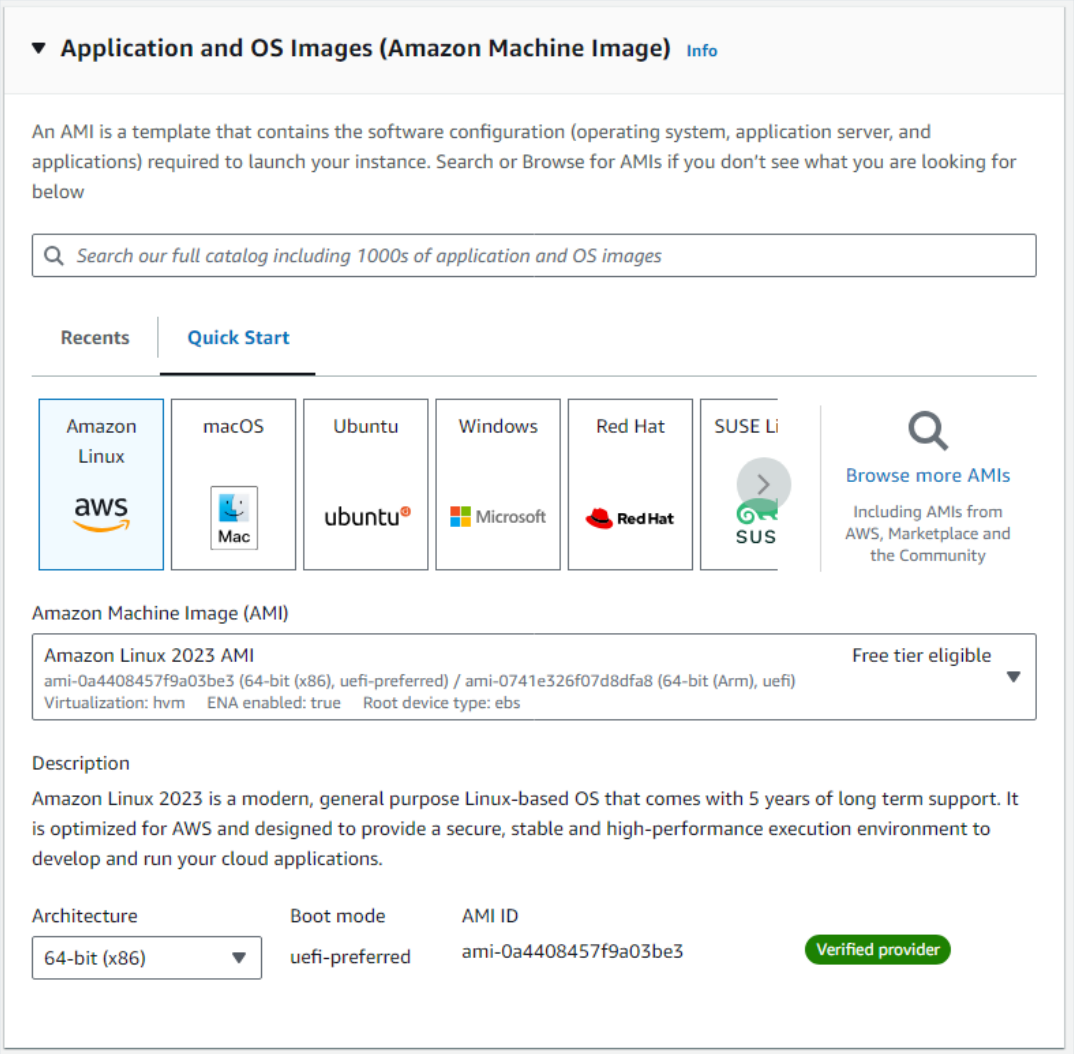
Train your first app server

Why bother?. The application should be highly scalable, so we use Auto Scaling groups. They create servers from launch templates, and a launch template needs an AMI. So we build one test server, configure the application on it, and save an AMI from it. The same idea is used for the web servers.

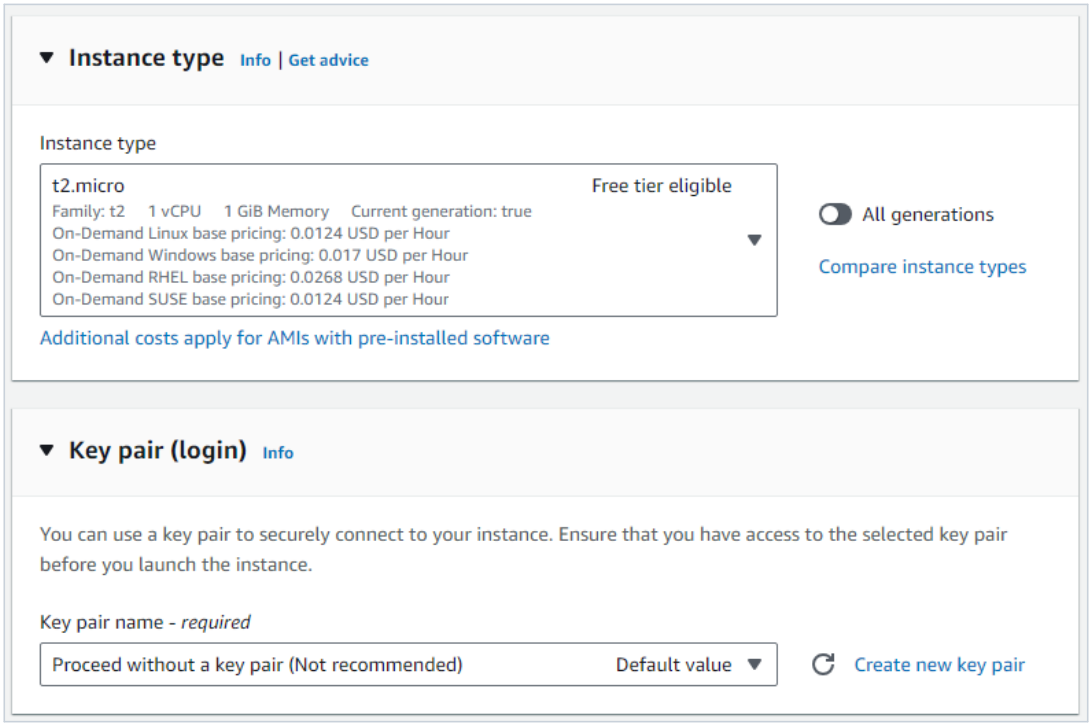
Search for **EC2**. Go to **Instances > Launch instances** and set it up as follows.



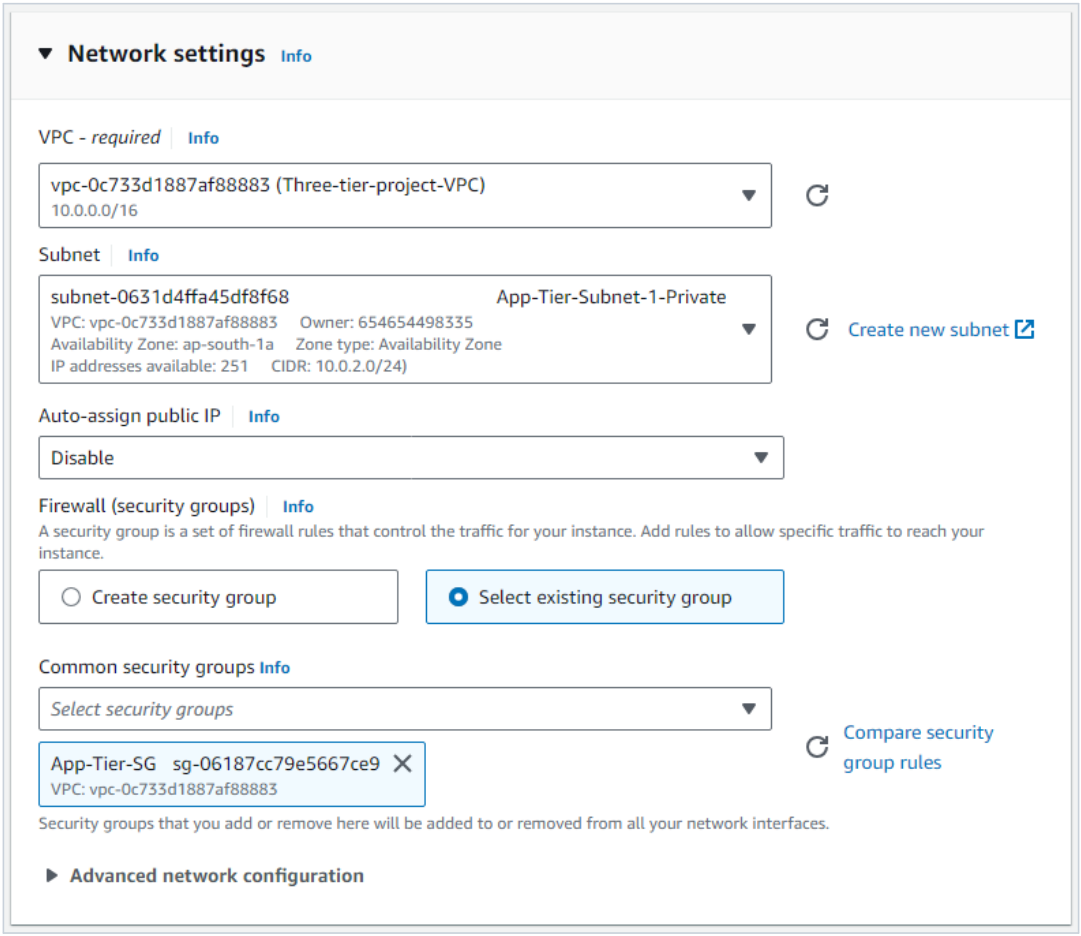
Name: Test-APP-Server.



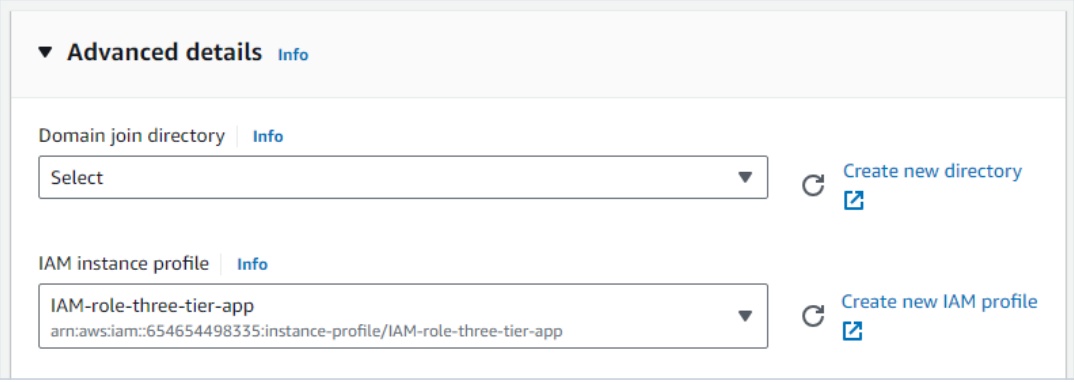
Amazon Machine Image: Amazon Linux 2023.



Instance type t2.micro, no key pair needed.



Network: your VPC, App-Tier-Subnet-1-Private, auto-assign public IP disabled, security group App-Tier-SG.

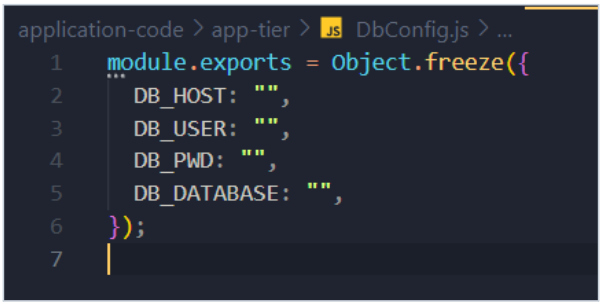


Advanced details: select your IAM instance profile.

Keep the other settings as default and click **Launch instance**. Next, connect to the server and configure the application tier.

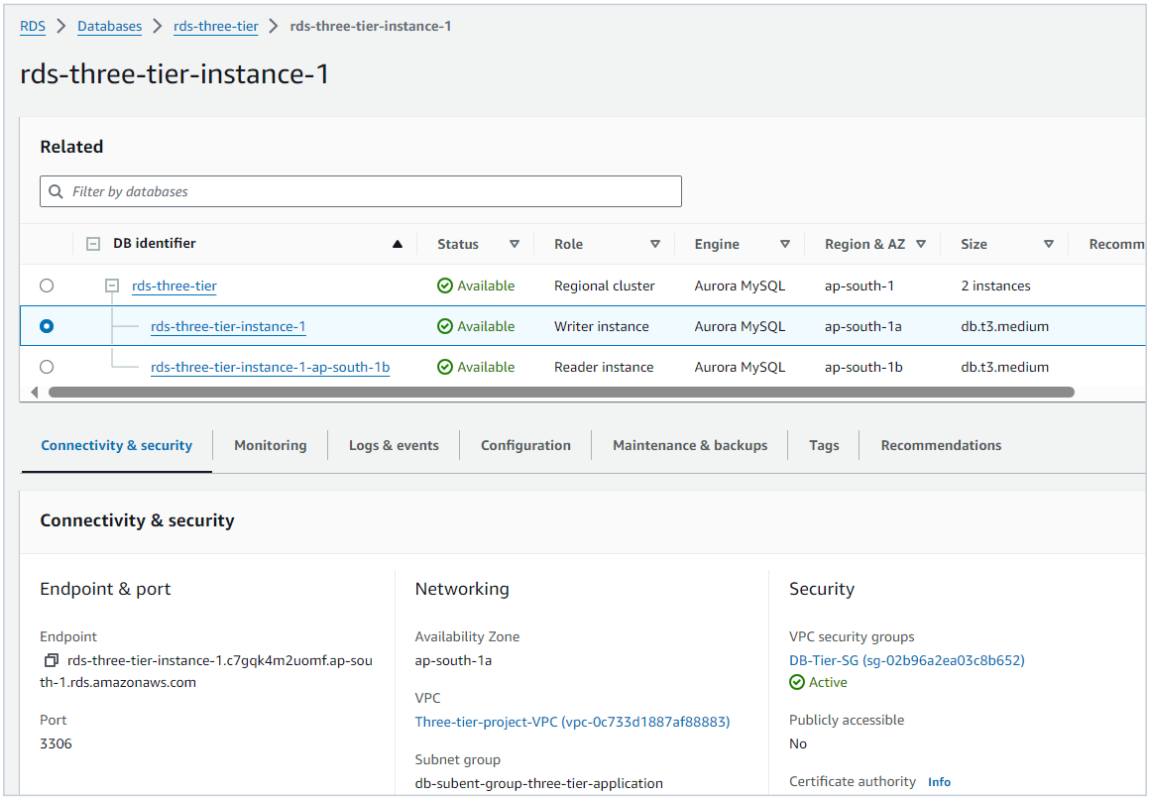
Add your database details to the code

On your PC, go to the source code folder: **application-code > app-tier**. Open **DbConfig.js** in your editor (the original guide uses VS Code). You need to change these values.



DbConfig.js before editing.

To get **DB_HOST**: search for **RDS > Databases**, select the **Writer instance**, and under Connectivity and security copy the **Endpoint**.



The Writer instance and its endpoint.

Setting	Value
DB_HOST	The Writer instance endpoint
DB_USER	The database username (admin)
DB_PWD	<YOUR_DB_PASSWORD> (the password you set)
DB_DATABASE	webappdb

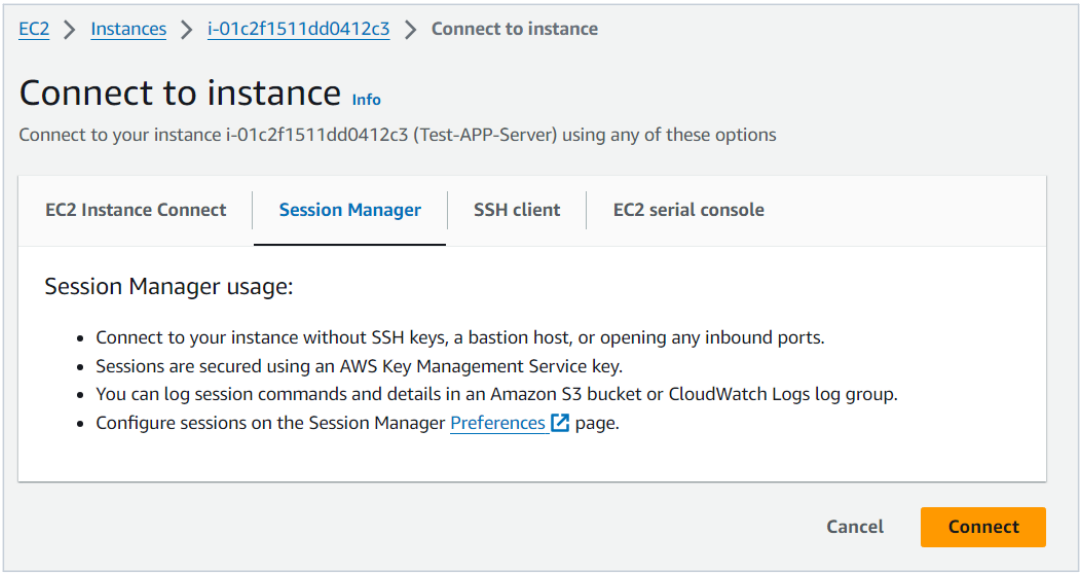


DbConfig.js after editing (the demo password is hidden here).

Now go to the S3 bucket. Open **application-code > app-tier**, delete the old **DbConfig.js** and upload your updated one.

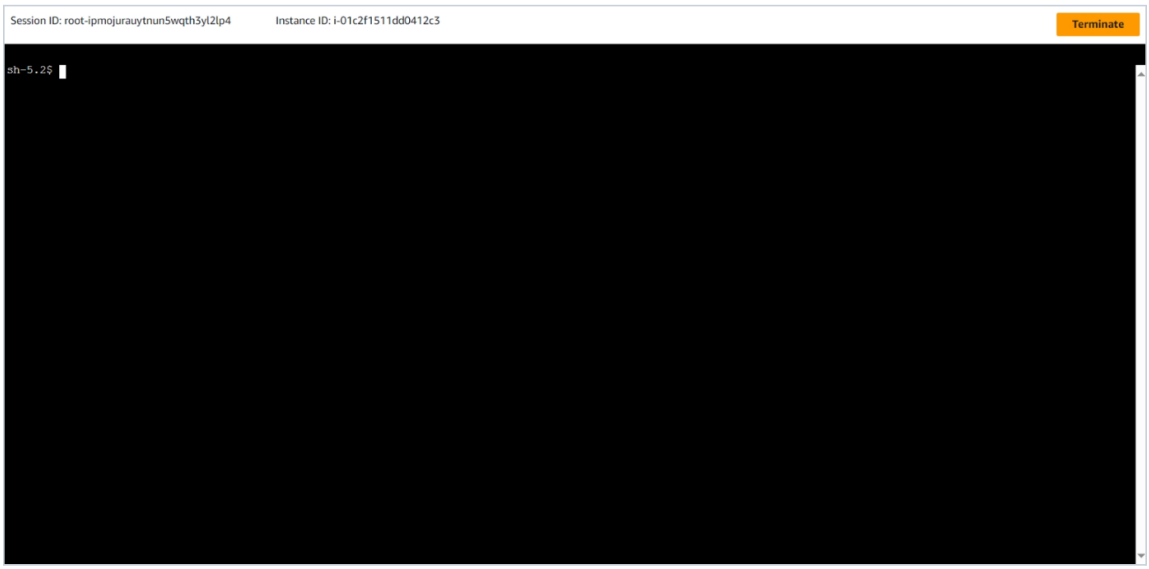
Connect to the server

Back in EC2, select **Test-APP-Server** and click **Connect**. Go to the **Session Manager** tab and click Connect.



Connect to instance: Session Manager tab.

A new tab opens like this. It means you are connected to the EC2 instance. The IAM role we created lets us connect even though the instance is in a private subnet.



The Session Manager terminal.

> **Pro tip.** Paste commands one block at a time and wait for each to finish before the next. If one fails, fix it before moving on.

Configure the app tier

First switch to the ec2-user:

```
sudo -su ec2-user
```

Install MySQL with these commands:

```
sudo wget https://dev.mysql.com/get/mysql80-community-release-el9-1.noarch.rpm
sudo dnf install mysql80-community-release-el9-1.noarch.rpm -y
sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2023
sudo dnf install mysql-community-client -y
```

Run `mysql --version` to confirm it installed correctly.

```
[ec2-user@ip-10-0-2-216 bin]$ mysql --version
mysql  Ver 8.0.39 for Linux on x86_64 (MySQL Community Server - GPL)
[ec2-user@ip-10-0-2-216 bin]$
```

MySQL client version output.

Connect to the database server. Use the Writer instance endpoint and the username **admin**:

```
mysql -h <RDS-Endpoint> -u admin -p
```

Press Enter and type your password. You will see this output.

```
[ec2-user@ip-10-0-2-216 bin]$ mysql -h rds-three-tier-instance-1.c7gqk4m2uomf.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 804
Server version: 8.0.32 Source distribution

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

A successful connection to the MySQL monitor.

Create the database:

```
CREATE DATABASE webappdb;
SHOW DATABASES;
USE webappdb;
```

```
mysql> create DATABASE webappdb;
Query OK, 1 row affected (0.00 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| webappdb           |
+--------------------+
5 rows in set (0.00 sec)

mysql> use webappdb;
Database changed
mysql>
```

The webappdb database is created and selected.

Create a table:

```
CREATE TABLE IF NOT EXISTS transactions(
  id INT NOT NULL AUTO_INCREMENT,
  amount DECIMAL(10,2),
  description VARCHAR(100),
  PRIMARY KEY(id));
```

```
mysql> CREATE TABLE IF NOT EXISTS transactions(id INT NOT NULL
    -> AUTO_INCREMENT, amount DECIMAL(10,2), description
    -> VARCHAR(100), PRIMARY KEY(id));
Query OK, 0 rows affected (0.02 sec)

mysql> show tables;
+--------------------+
| Tables_in_webappdb |
+--------------------+
| transactions       |
+--------------------+
1 row in set (0.00 sec)

mysql>
```

The transactions table is created.

Insert a sample value, then type `exit` and press Enter:

```
INSERT INTO transactions (amount,description) VALUES ('400','groceries');
SELECT * FROM transactions;
exit
```

```
mysql> INSERT INTO transactions (amount,description) VALUES ('400','groceries');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM transactions;
+----+--------+-------------+
| id | amount | description |
+----+--------+-------------+
|  1 | 400.00 | groceries   |
+----+--------+-------------+
1 row in set (0.00 sec)

mysql>
```

The sample row (400.00, groceries) is stored.

Next, copy the application tier code from the S3 bucket. Replace <YOUR-S3-BUCKET-NAME> with your bucket name.

```
cd /home/ec2-user
sudo aws s3 cp s3://<YOUR-S3-BUCKET-NAME>/application-code/app-tier app-tier --recursive
cd app-tier
sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
sudo chmod -R 755 /home/ec2-user/app-tier
```

```
[ec2-user@ip-10-0-2-216 bin]$ cd /home/ec2-user
[ec2-user@ip-10-0-2-216 ~]$ sudo aws s3 cp s3://s3-three-tier-app-rutuja/applicatio
n-code/app-tier app-tier --recursive
download: s3://s3-three-tier-app-rutuja/application-code/app-tier/DbConfig.js to ap
p-tier/DbConfig.js
download: s3://s3-three-tier-app-rutuja/application-code/app-tier/package.json to a
pp-tier/package.json
download: s3://s3-three-tier-app-rutuja/application-code/app-tier/README.md to app-
tier/README.md
download: s3://s3-three-tier-app-rutuja/application-code/app-tier/index.js to app-t
ier/index.js
download: s3://s3-three-tier-app-rutuja/application-code/app-tier/TransactionServic
e.js to app-tier/TransactionService.js
download: s3://s3-three-tier-app-rutuja/application-code/app-tier/package-lock.json
 to app-tier/package-lock.json
[ec2-user@ip-10-0-2-216 ~]$ cd app-tier
[ec2-user@ip-10-0-2-216 app-tier]$ ls
DbConfig.js  TransactionService.js  package-lock.json
README.md    index.js               package.json
[ec2-user@ip-10-0-2-216 app-tier]$ sudo chown -R ec2-user:ec2-user /home/ec2-user/a
pp-tier
[ec2-user@ip-10-0-2-216 app-tier]$ sudo chmod -R 755 /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-216 app-tier]$
```

The app-tier files downloaded from S3.

Now install Node.js:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 16
nvm use 16
npm install -g pm2
npm install
npm audit fix
```

Then run index.js and save the configuration:

```
pm2 start index.js
pm2 logs
```

```
m2/dump.pm2
PM2        | 2024-08-15T18:42:55: PM2 log: Concurrent actions   : 2
PM2        | 2024-08-15T18:42:55: PM2 log: SIGTERM timeout      : 1600
PM2        | 2024-08-15T18:42:55: PM2 log: ================================
========================================
PM2        | 2024-08-15T18:42:55: PM2 log: App [index:0] starting in -fork mode-
PM2        | 2024-08-15T18:42:55: PM2 log: App [index:0] online

/home/ec2-user/.pm2/logs/index-error.log last 15 lines:
/home/ec2-user/.pm2/logs/index-out.log last 15 lines:
0|index    | AB3 backend app listening at http://localhost:4000
```

The backend app listening on port 4000.

If you see output like this, the backend is listening on port 4000. Press **Ctrl+C** to exit the log view. Then run these commands so the app restarts after a reboot, and test the health check:

```
pm2 startup
sudo env PATH=$PATH:/home/ec2-user/.nvm/versions/node/v16.20.2/bin \
  /home/ec2-user/.nvm/versions/node/v16.20.2/lib/node_modules/pm2/bin/pm2 startup systemd \
  -u ec2-user --hp /home/ec2-user
pm2 save
curl http://localhost:4000/health
```

```
[PM2] Remove init script via:
$ pm2 unstartup systemd
[ec2-user@ip-10-0-2-216 app-tier]$ pm2 save
rrent configuration)
[PM2] Saving current process list...
[PM2] Successfully saved in /home/ec2-user/.pm2/dump.pm2
[ec2-user@ip-10-0-2-216 app-tier]$ curl http://localhost:4000/heal
alth check)
"This is the health check"[ec2-user@ip-10-0-2-216 app-tier]$
```

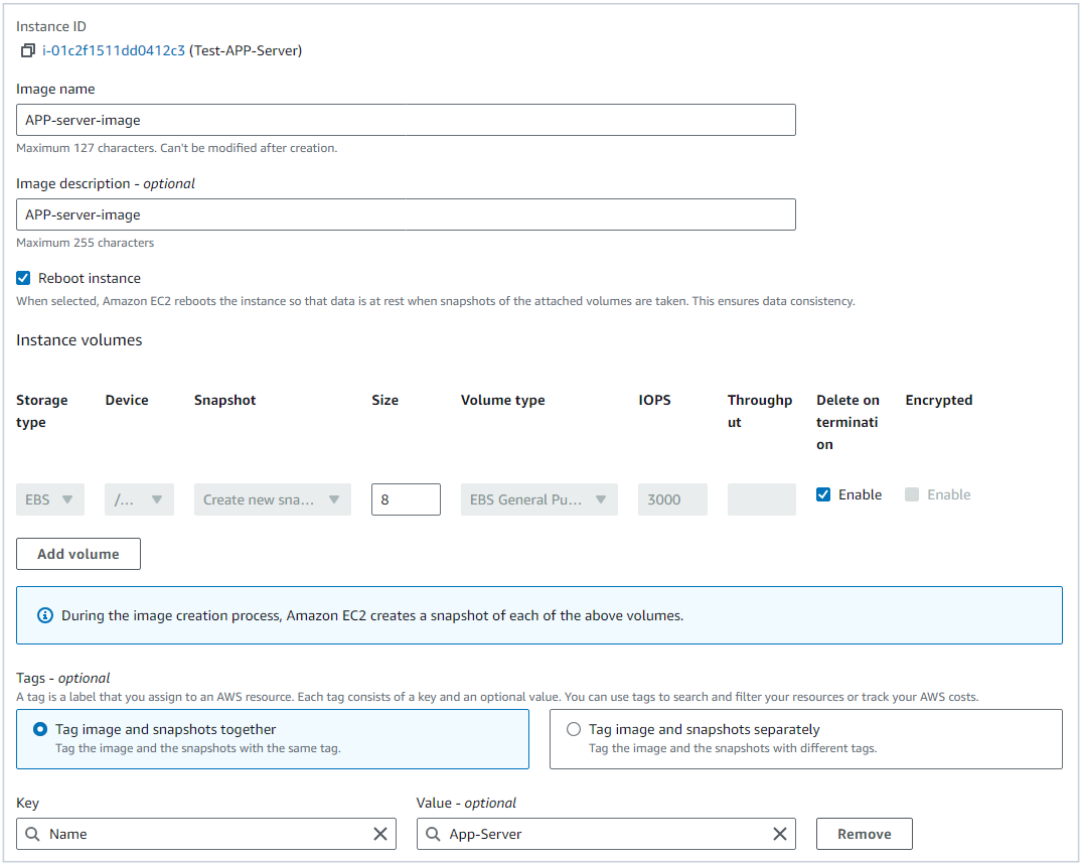
pm2 saved, and the health check replies "This is the health check".

Checkpoint. The health check returns a healthy reply. You successfully set up the app server.

8 Create an AMI of the app server

Freeze it so it can be cloned

Next, create an image from the Test-APP-Server instance. Go to **Instances**, choose Test-APP-Server and go to **Actions > Image and templates > Create image**.



Create image: name it APP-server-image.

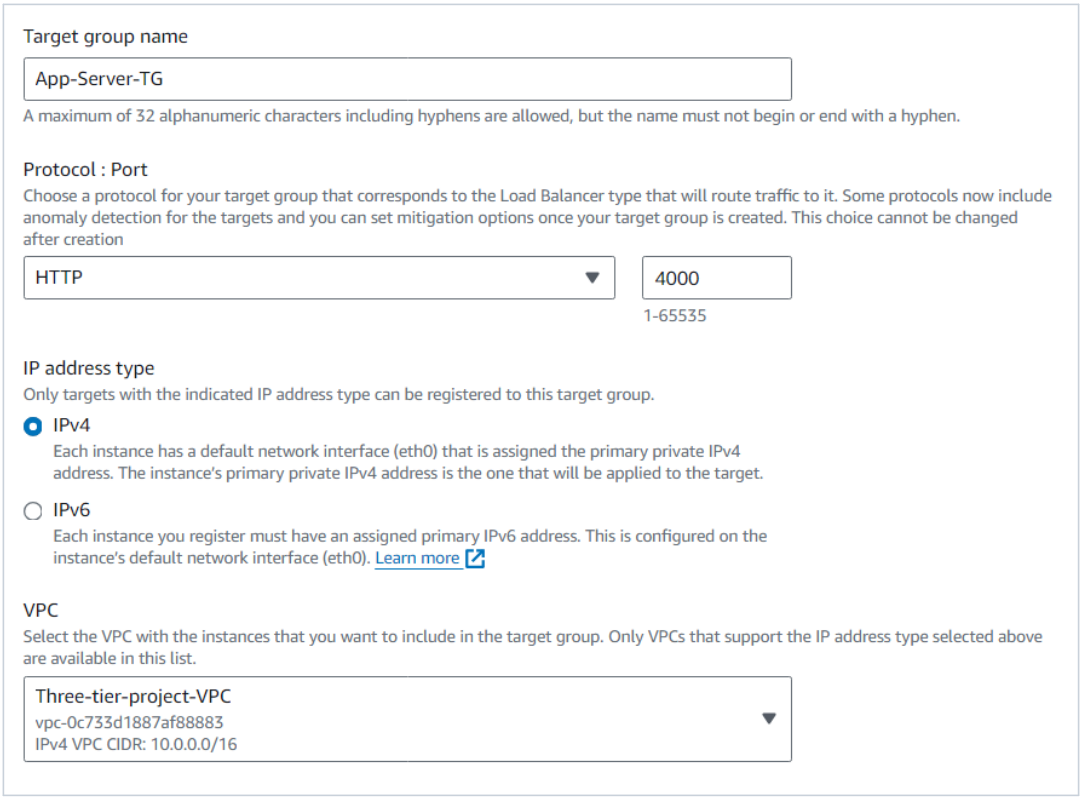
Open the **AMIs** section. You can see the App-Server AMI. Wait until its status is Available.

9 Create the target group, launch template and internal load balancer (app tier)

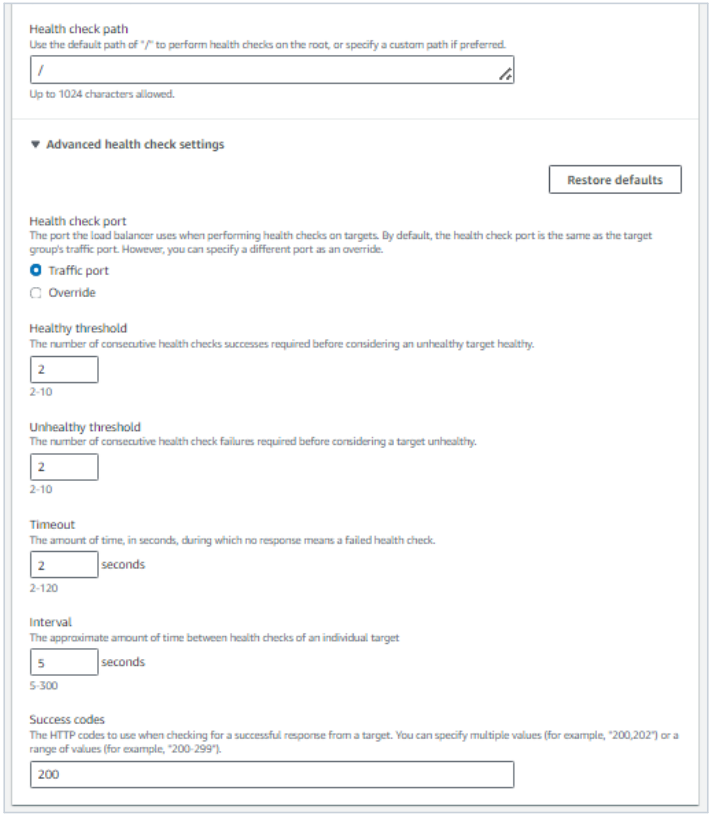
Hire the traffic cop for the app tier

Create the target group

Go to **EC2 > Target Groups > Create target group** and change only the following settings: protocol HTTP, port **4000**, your VPC, and health check path **/health**. Click Next, then Create target group.



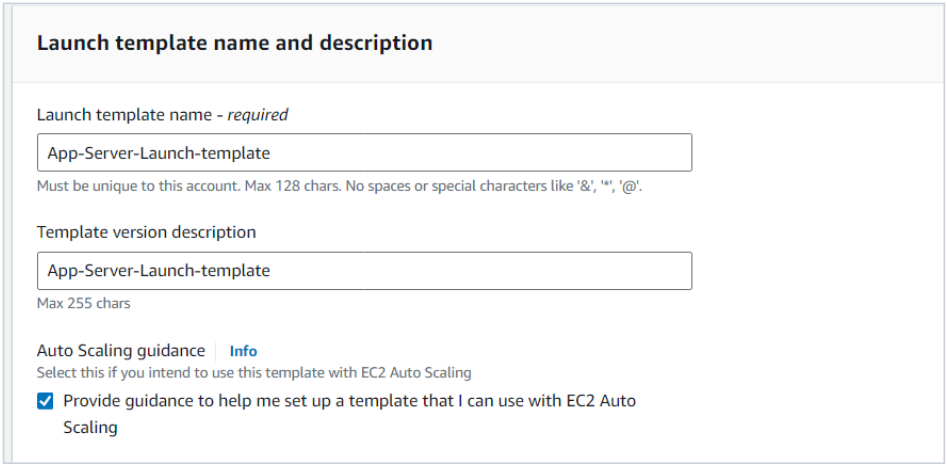
Target group App-Server-TG: HTTP on port 4000, in your VPC.



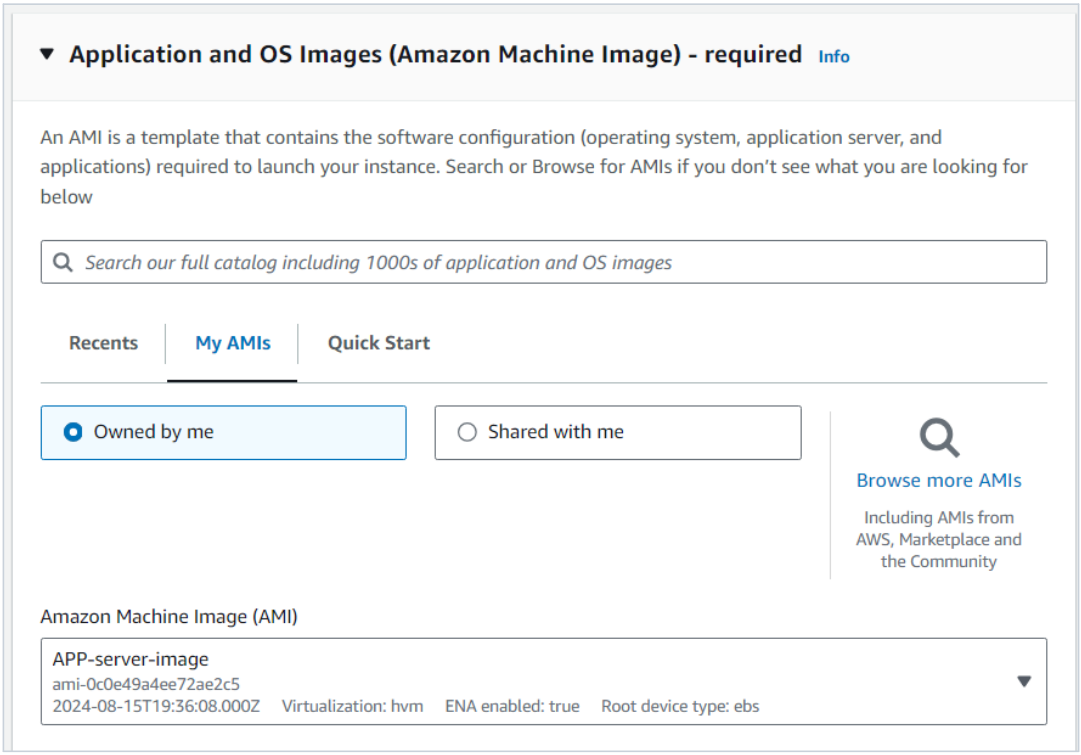
Health check path set to /health.

Create the launch template

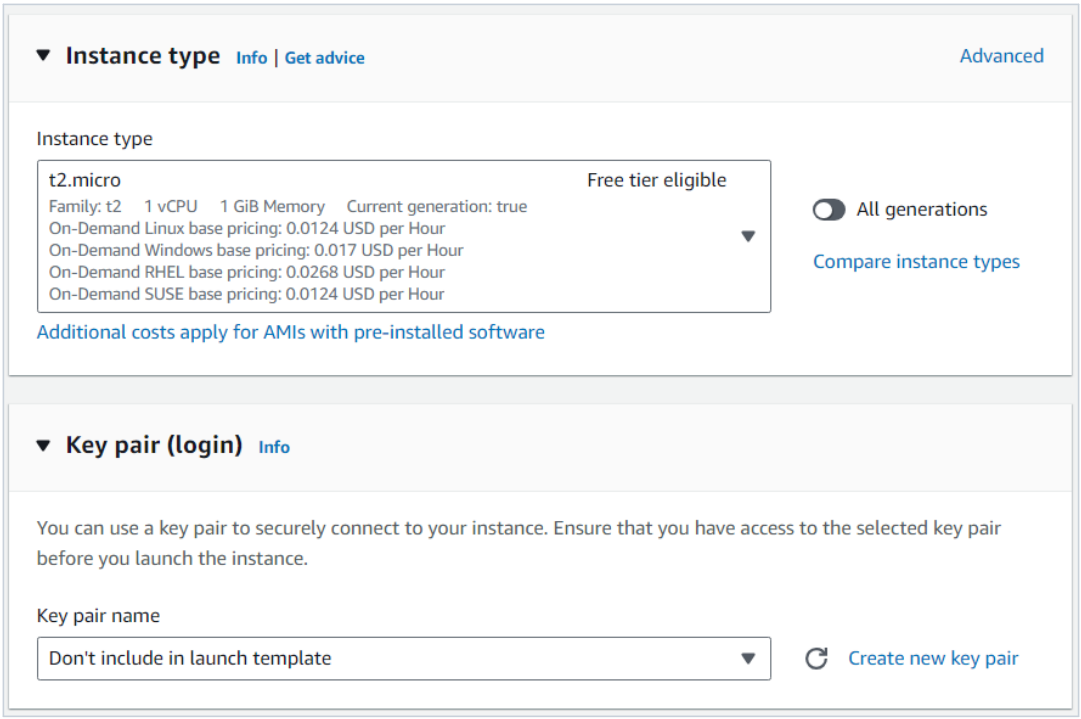
Go to **EC2 > Launch Templates > Create launch template**.



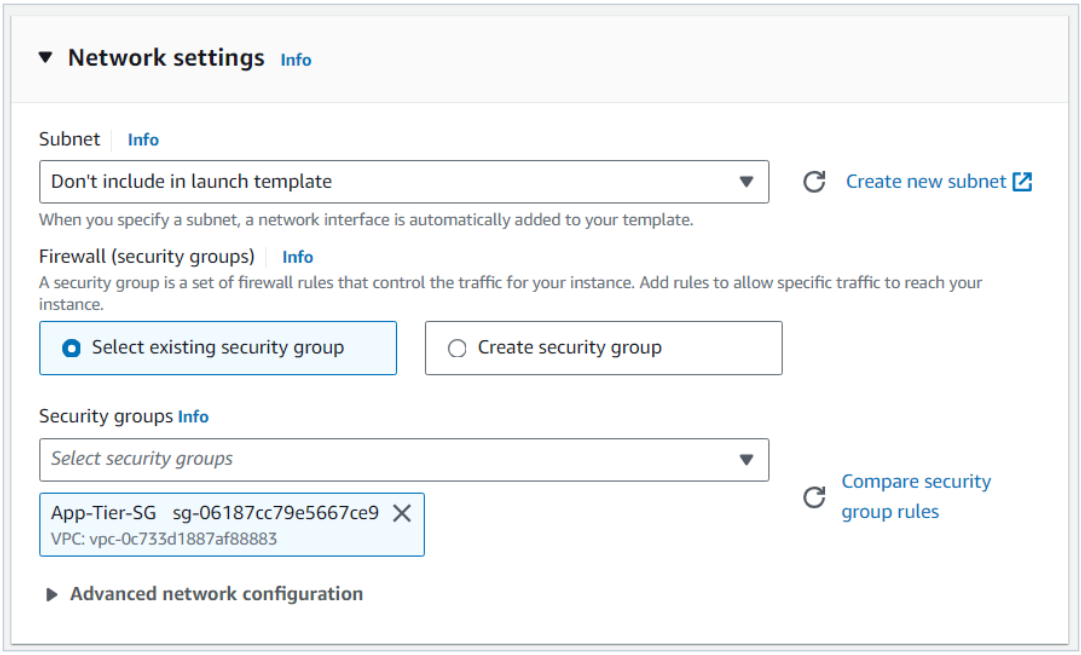
Launch template name and description.



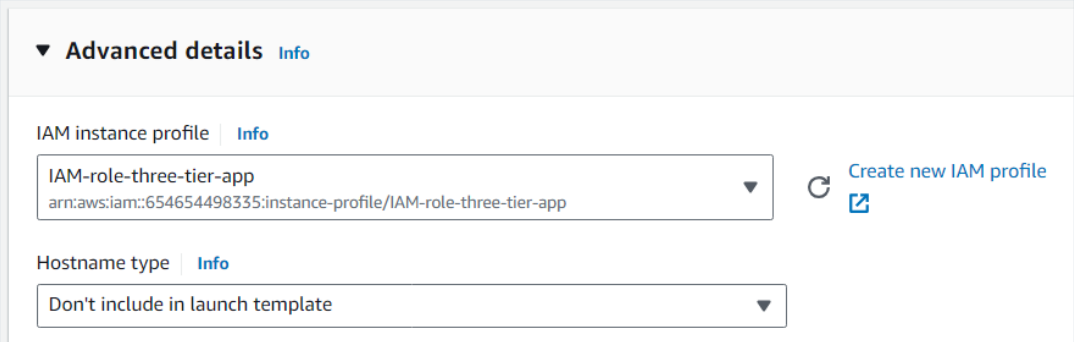
AMI: choose your APP-server-image under My AMIs.



Instance type t2.micro. Key pair: Don't include in launch template.



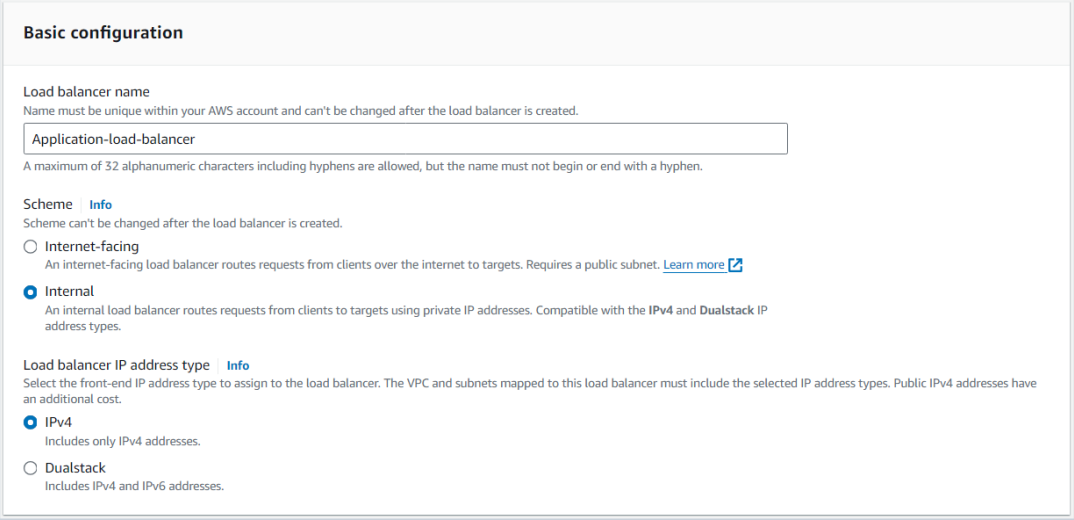
Network settings: security group App-Tier-SG.



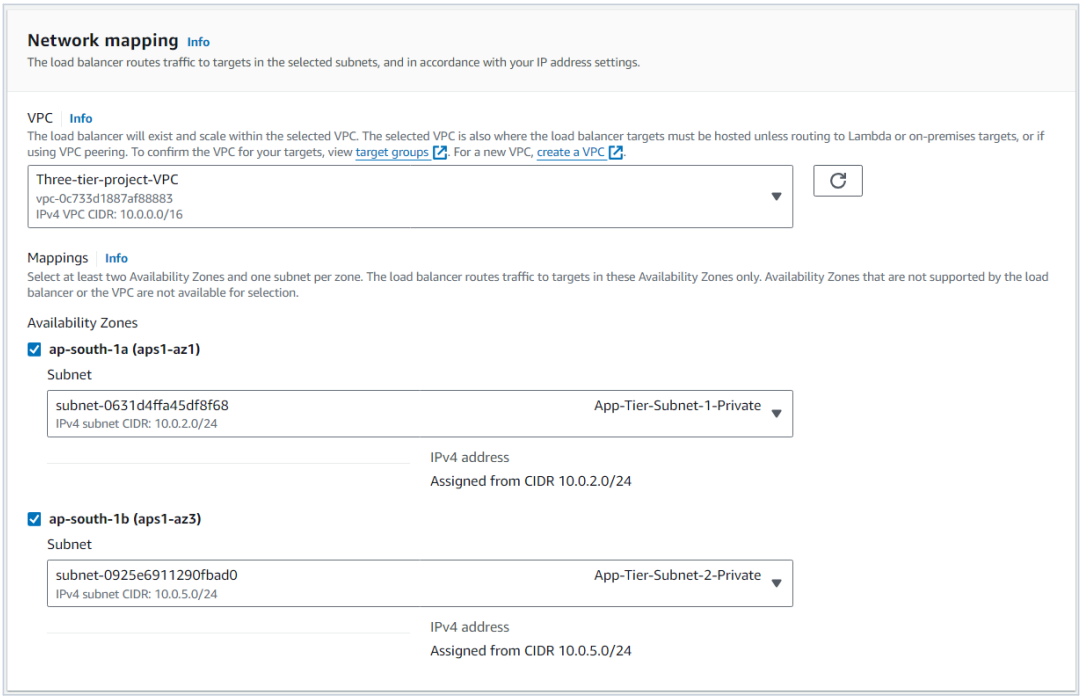
Advanced details: your IAM instance profile.

Create the application load balancer (internal)

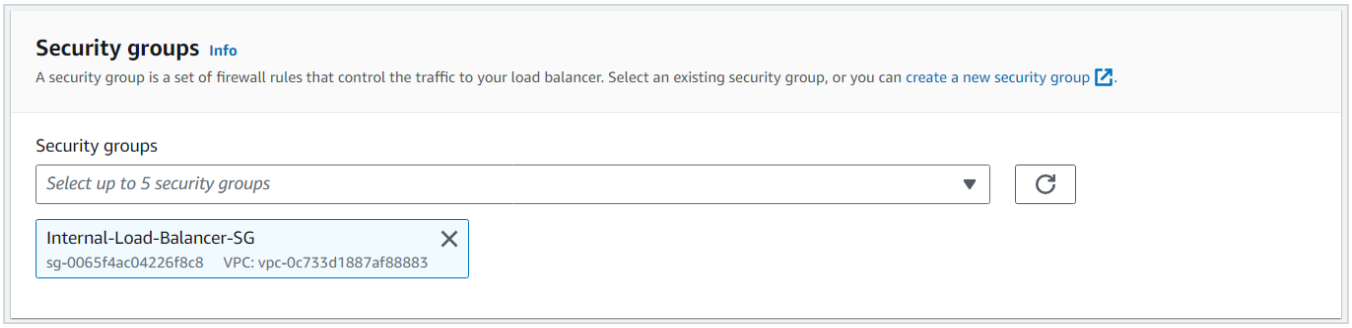
Go to **EC2 > Load Balancers > Create load balancer** and create an **Application Load Balancer**.



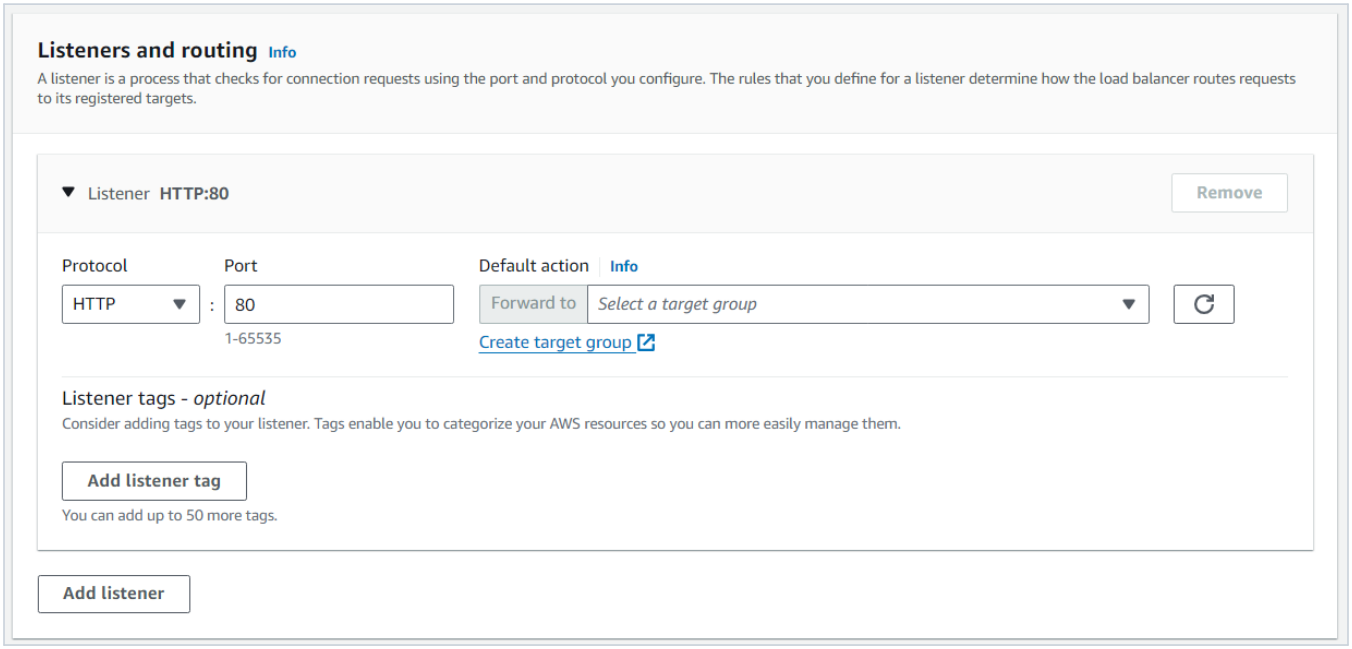
Basic configuration: scheme Internal.



Network mapping: your VPC, and the two private app subnets in 1a and 1b.



Security group: Internal-Load-Balancer-SG.



Listener HTTP 80 forwarding to App-Server-TG.

10 Create the test web server and set it up

Meet the web server

Follow the same process as for the app server. Only the EC2 configuration part is different. First go to the source code folder on your PC and open **nginx.conf** in an editor. Then go to the internal load balancer you created and copy its **DNS name**. Paste it into this section of nginx.conf.

```
#proxy for internal lb
location /api/{
        proxy_pass http://<Internal load balancer DNS name>:80/;
}
```

The placeholder in nginx.conf.

```
#proxy for internal lb
location /api/{
        proxy_pass http://internal-Application-load-balancer-1096585234.ap-south-1.elb.amazonaws.com:80/;
}
```

nginx.conf with the internal load balancer DNS name pasted in.

Go to the S3 bucket, delete the old nginx.conf and upload the new one.

Now go to **EC2 > Launch instance** and create an instance for the test web server. Change the name (**Test-Web-Server**), the VPC, the subnet (**Web-Tier-Subnet-1-Public**), the security group (**Web-Tier-SG**), and select the IAM role under Advanced details. Launch the instance. Then connect to it and configure it as follows.

Switch users:

```
sudo -su ec2-user
cd /home/ec2-user
```

Copy the content from the S3 bucket:

```
sudo aws s3 cp s3://<S3_BUCKET_NAME>/application-code/web-tier web-tier --recursive
sudo chown -R ec2-user:ec2-user /home/ec2-user
sudo chmod -R 755 /home/ec2-user
```

Install Node.js:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 16
nvm use 16
cd /home/ec2-user/web-tier
npm install
```

Build the app:

```
cd /home/ec2-user/web-tier
```

```
npm run build
```

Install Nginx (the web server):

```
sudo yum install nginx -y
cd /etc/nginx
sudo mv nginx.conf nginx-backup.conf
sudo aws s3 cp s3://<S3_BUCKET_NAME>/application-code/nginx.conf .
sudo chmod -R 755 /home/ec2-user
sudo service nginx restart
sudo chkconfig nginx on
```

Step 11 of 15 | Build the tiers

11 Create an AMI of the web server

Make a photocopy of it

Go to **Instances** and select **Test-Web-Server**. Then go to **Actions > Image and templates > Create image**. Name it **Web-Server-Image** and wait until it shows Available under AMIs.

Step 12 of 15 | Build the tiers

12 Create the target group, launch template and external load balancer (web tier)

Open the front door to the world

Same process as Step 9, with these changes.

Item	Settings
Target group	Change Protocol : Port to 80. Select your VPC, give it a name, and create it.
Launch template	Choose Web-Server-Image for the AMI and Web-Tier-SG for the security group. Other settings are the same as for the app tier.
External load balancer	Scheme: Internet-facing. Choose your VPC. Choose the Availability Zones of the web tier. Choose the External-Load-Balancer-SG security group. Choose the web target group under Listeners and routing. Create the load balancer.

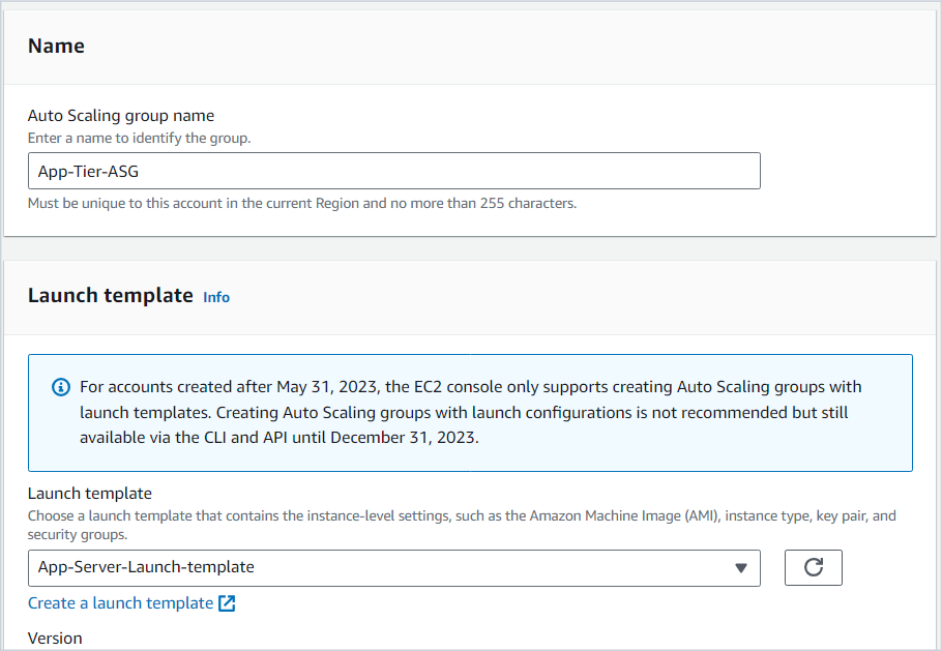
Level up. Level 3 complete. Both tiers have templates and load balancers. One more push: let AWS run them for you.

13 Create the Auto Scaling groups

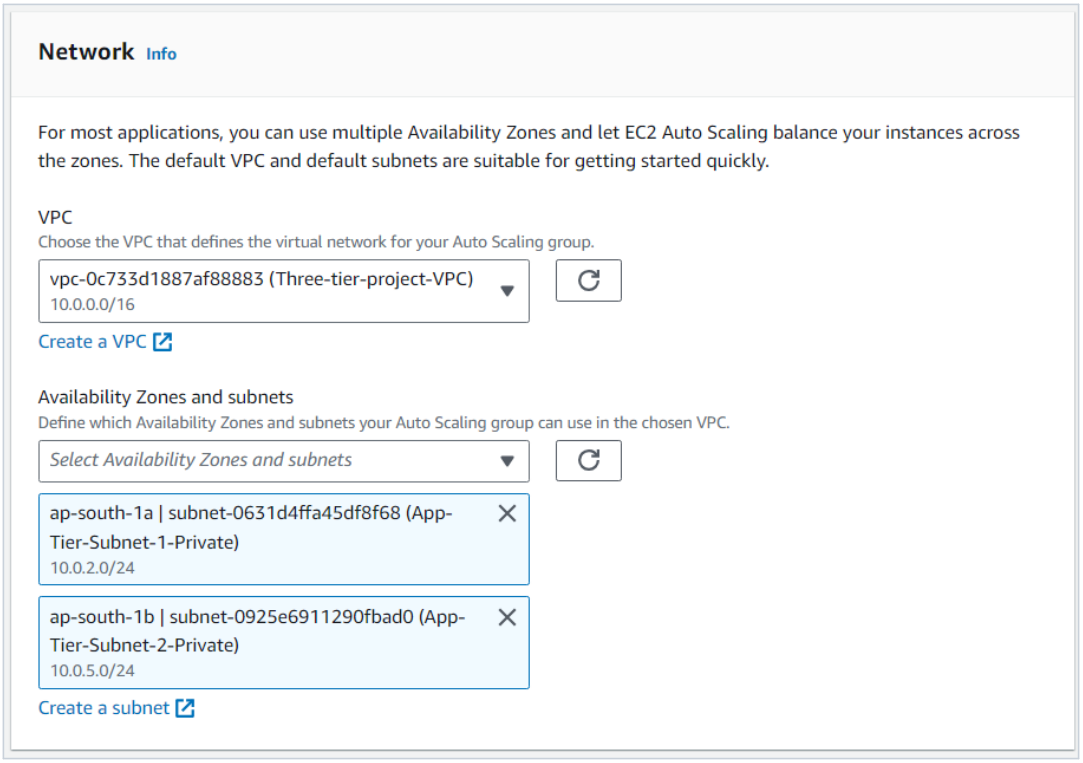
Let AWS do the heavy lifting

Auto Scaling group for the app tier

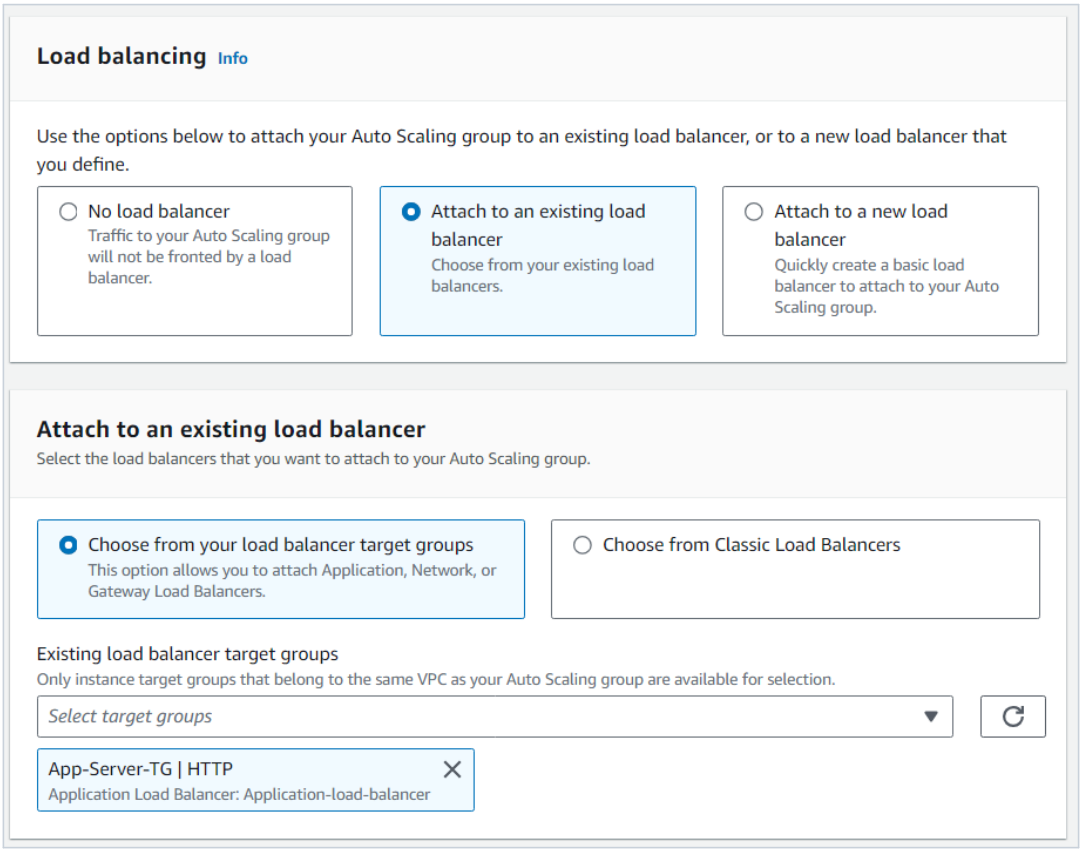
Go to **EC2 > Create Auto Scaling group**.



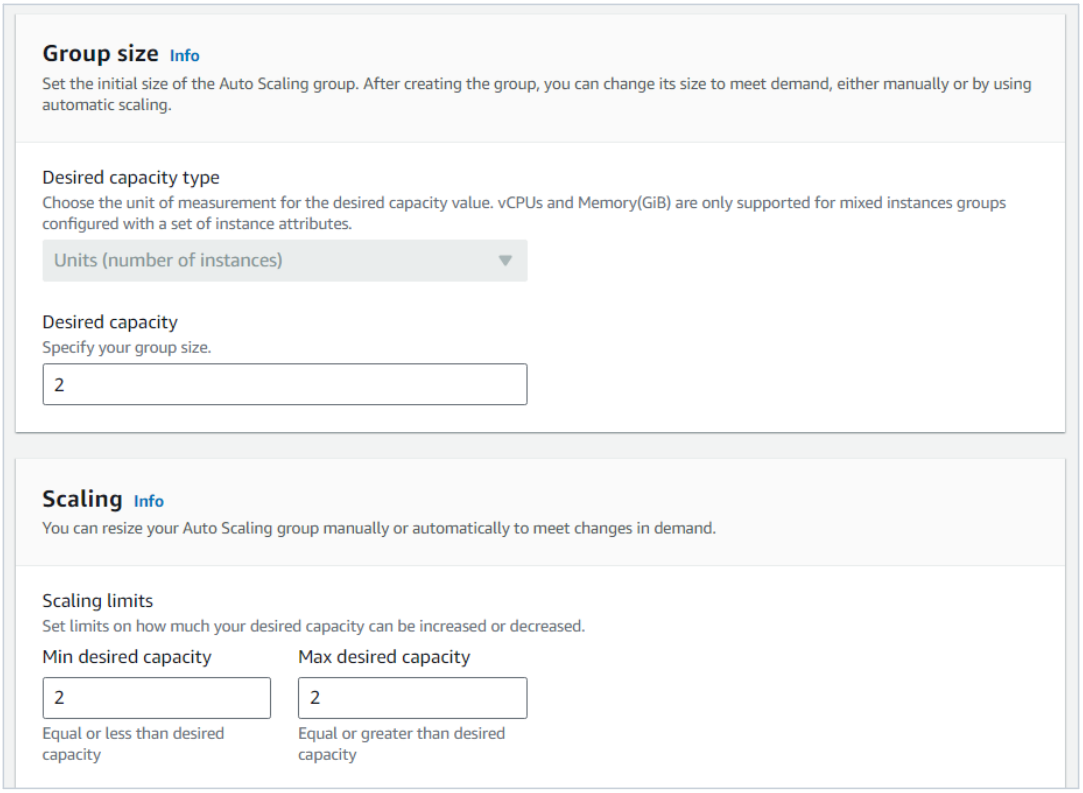
Name: App-Tier-ASG, and the app launch template.



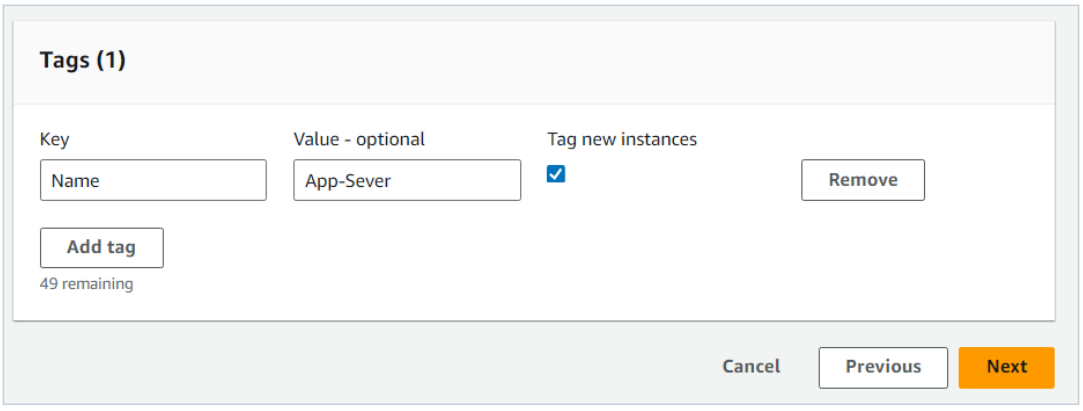
Network: your VPC and the two private app subnets.



Load balancing: attach to the existing target group.

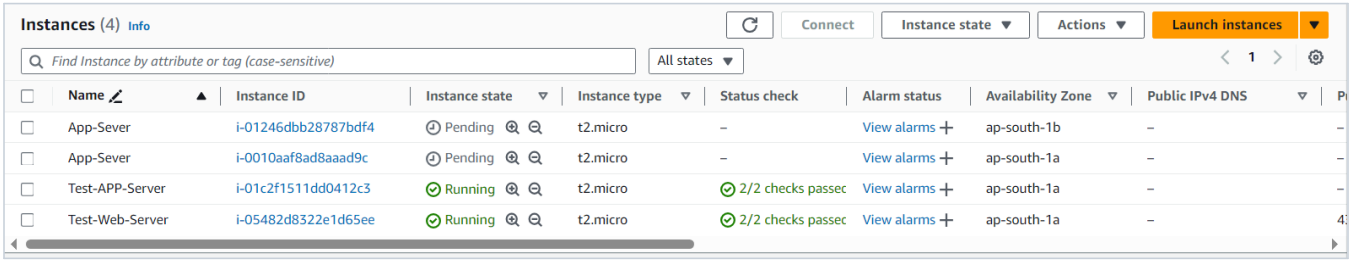


Group size: desired, minimum and maximum capacity all set to 2.



Tags: Name = App-Sever, tag new instances. Click Next.

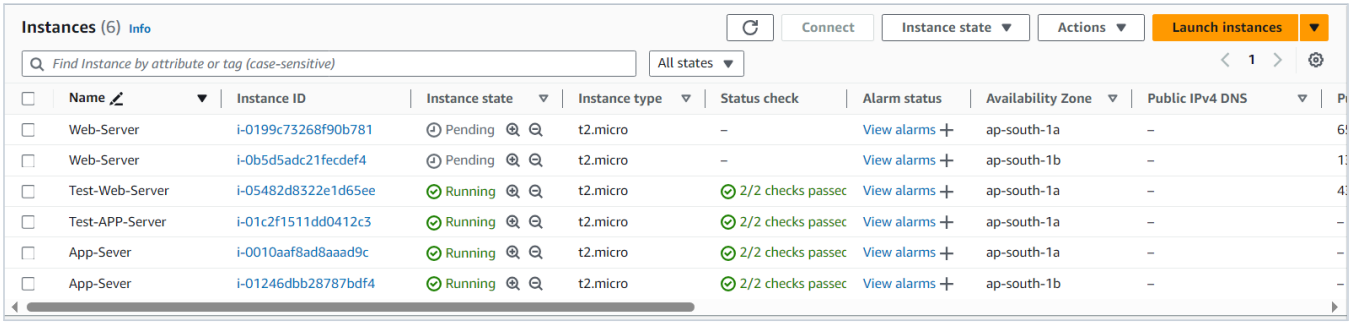
Keep the other settings as default and create the Auto Scaling group. If you check the instance list, you can see two app servers being created.



Two App-Sever instances starting, next to the two test servers.

Auto Scaling group for the web tier

The process is the same. Choose the components related to the web tier: the web launch template, the two public web subnets and the web target group, with a desired capacity of 2. After it is created you can see the four new instances and the test instances in the list. Now you can delete the test instances.



Two web servers and two app servers launched by Auto Scaling, next to the two test servers.

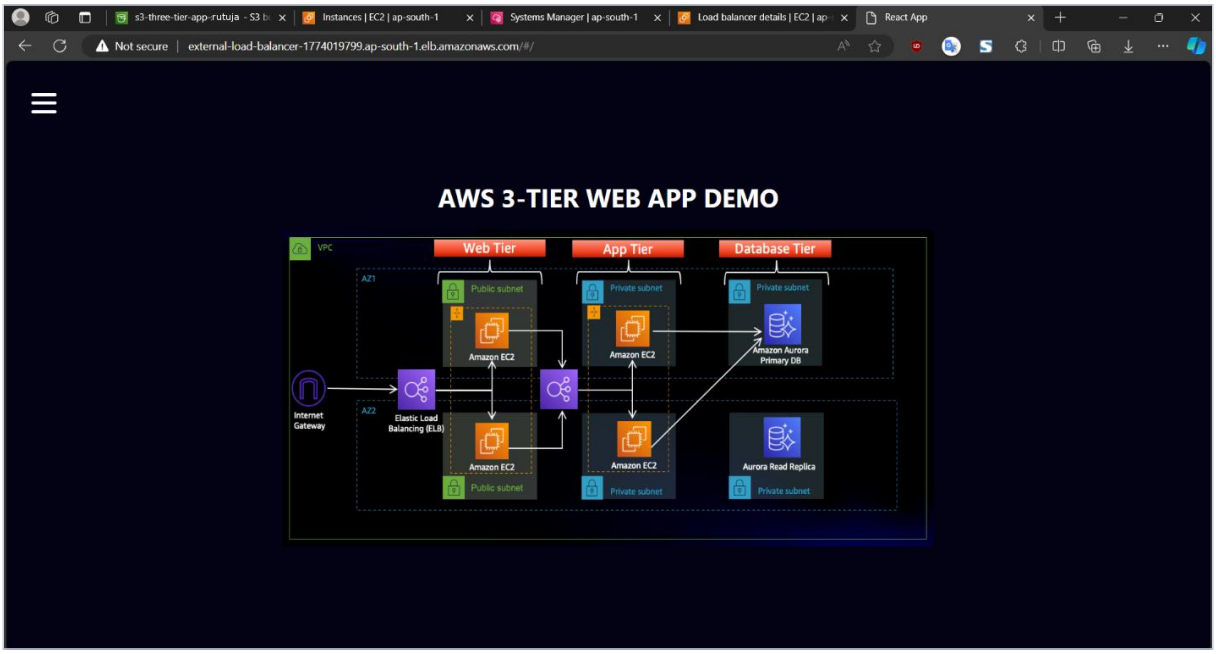
Checkpoint. Four new servers (2 app, 2 web) are running. You did a great job. Now you can access the web application.

Step 14 of 15 | Go live

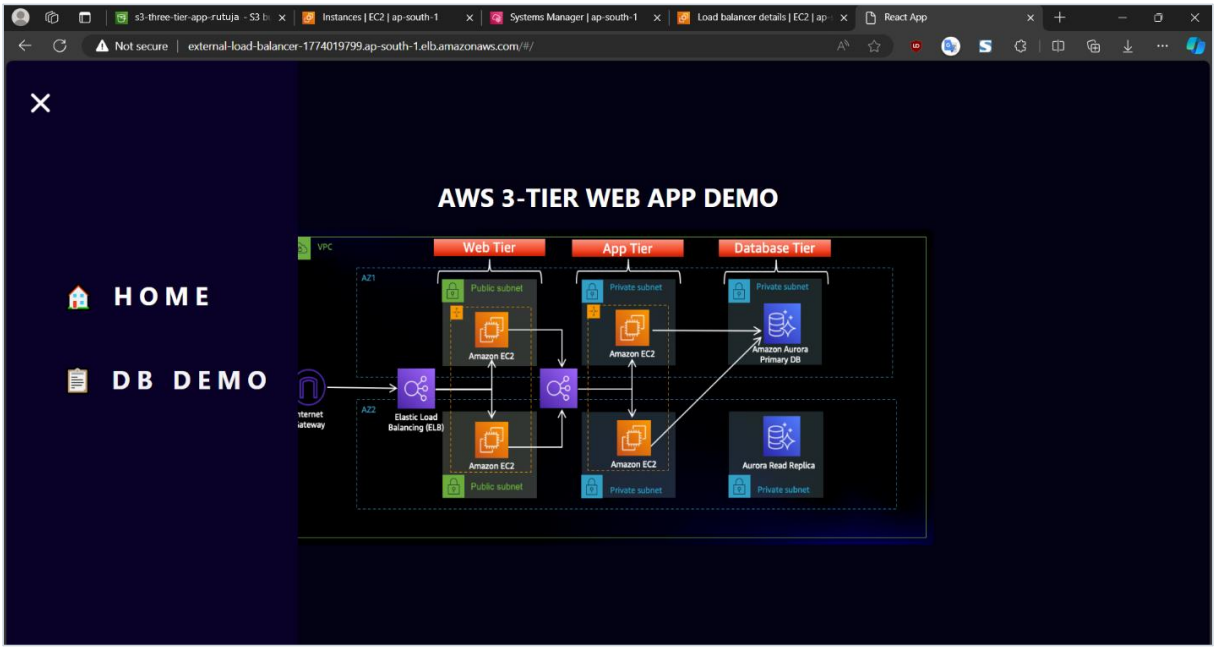
14 Access the application

The moment of truth

Go to **EC2 > Load Balancers**. Select the **external load balancer** and copy its DNS name. Paste it into your browser. You will see the application running.

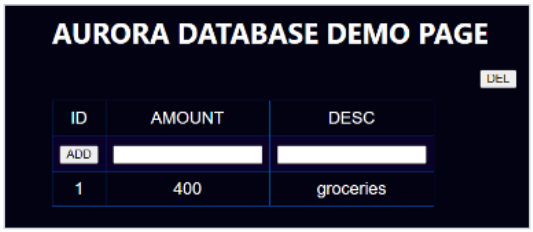


The application home page, served through the external load balancer.



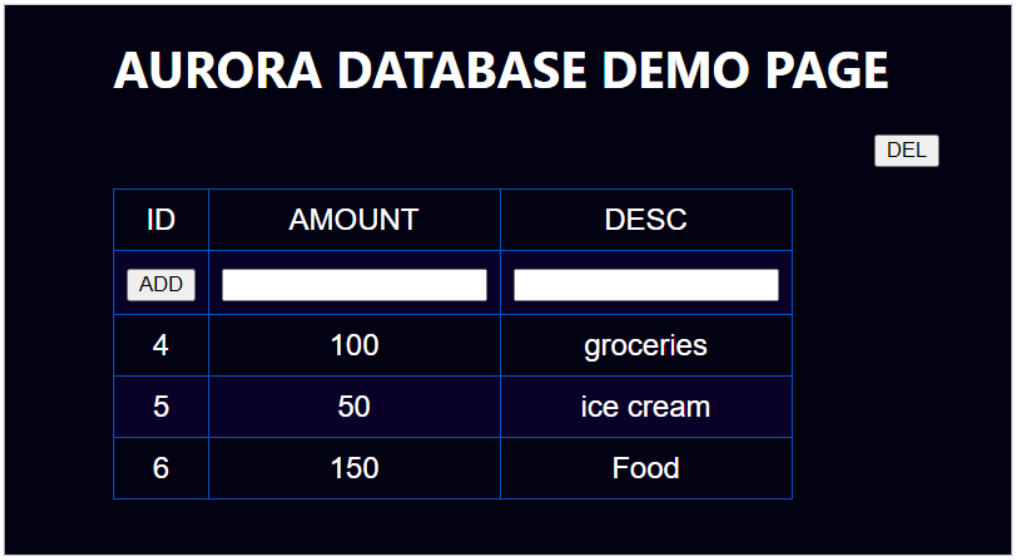
The menu: HOME and DB DEMO.

Go to the **DB DEMO** section and check whether the database is working. You can see the example data we entered when we set up the database, which means the application is receiving data from it.



DB DEMO page showing the sample row from the database.

You can add and delete items.



Adding and deleting items on the Aurora Database Demo page.

Pro tip. If the page does not load, open it in an Incognito window and make sure the address starts with http, not https.

Checkpoint. Congratulations! You have successfully implemented a three-tier web application on AWS.

Note. Here the external load balancer's DNS name is used as the URL, which is not user friendly. You can buy a domain name from a domain name service and set it up with Route 53 to access the website with that domain name.

Step 15 of 15 | Go live

15 Clean up the infrastructure

Leave no trace (and no bill)

Some of the services we used charge money, so delete the demo project when you finish. Delete things in **this order**, because AWS blocks deleting anything that is still in use.

1. Auto Scaling groups
2. Load balancers (delete the listeners first, then the load balancers)
3. Target groups
4. Launch templates
5. AMIs
6. Snapshots
7. EC2 instances
8. Aurora database (first go to Modify and disable delete protection. When deleting, do not create a final snapshot, and tick I acknowledge)
9. NAT gateways
10. Elastic IPs (release them)
11. Route tables (remove all subnet associations first)
12. Internet gateways (detach, then delete)
13. VPC
14. S3 bucket (empty it first)
15. IAM role

Quick troubleshooting

Problem	Likely fix
Session Manager will not connect	Attach the IAM role to the instance, and check the private subnet has a NAT route.
aws s3 cp fails	Check the bucket name and that the role has S3 read access.
App cannot reach the database	Check DB_HOST, user and password in DbConfig.js, and that DB-Tier-SG allows App-Tier-SG on port 3306.
Load balancer shows unhealthy targets	Check the port, the /health path and the security groups.
Website shows 502 or 504	The internal load balancer DNS name in nginx.conf is probably wrong.
Site does not load in the browser	Try an Incognito window, and confirm the external load balancer allows port 80.

You built it.

A secure, scalable, three-tier application is now running on AWS: a public web tier, a private app tier and a protected database, spread across two Availability Zones. Keep the diagram, keep your notes, and remember to clean up so the bill stays tiny.

Created by Rutuja Naik
LinkedIn: linkedin.com/in/rutuja-naik07/
Source code: github.com/Rutuja-Naik07/Project-Three-Tier-Architecture-on-AWS